# Lecture Notes, Part 1

Galerkin Projection for PDEs: spectral methods, FEM and POD-Galerkin

M. A. Mendez

## Contents

**1 Section 2: approximation, inner products, projection** **3**
Slide 10–11: Normal equations, in vector form . . . 3
Slide 13: Solving the normal equations with Cholesky . . . 4
Slide 12: Gram–Schmidt . . . 5
Slide 16: Projection in function space . . . 6
Slide 18: Discrete inner products: from integrals to matrices . . . 7
Slide 21: Two errors, and only one is our fault . . . 7

**2 Section 3: Galerkin projection of a PDE** **9**
Slide 29: The Galerkin condition . . . 9
Slide 29b: The Galerkin condition in the continuous setting . . . 9
Slide 30–31: From the PDE to a system of ODEs . . . 10
Slide 32–33: The quadratic tensor, written out . . . 11
Slide 54: Self-adjoint $\mathcal{L}$ gives symmetric $\boldsymbol{K}$, and skew gives antisymmetric . . . 12
Slide 50–51: Petrov–Galerkin, and why it stabilises . . . 14
Slide 40: The weak form and integration by parts . . . 15
Slide 43: Spectral Galerkin: why the operators collapse . . . 16
Slide 44a: Why finite elements need the weak form . . . 16
Slide 44b: The hat function, written out . . . 17
Slide 45a: The reference element and the change of variables . . . 18
Slide 45: The finite element matrices, assembled by hand . . . 18
Slide 45b: Assembly, with the indices written out . . . 19
Slide 45c: The convective and the quadratic terms, element by element . . . 20
Slide 50: The lifting, done term by term . . . 20
Slide 52: The projection of the tutorial problem . . . 21

**3 Section 4: POD and POD-Galerkin** **23**
Slide 58: From the optimisation problem to the eigenproblem . . . 23
Slide 59: From the kernel to a matrix: what the quadrature does . . . 26
Slide 60: The method of snapshots, derived rather than quoted . . . 27
Slide 62: Optimality: what Eckart–Young actually says . . . 27
Slide 75: Projecting the tutorial problem onto POD modes, at $n_r = 3$ . . . 28
Slide 76: Counting the online cost . . . 29
Slide 110: Merging bases from several parameter classes . . . 29

**4 Merging bases from several classes, slowly** **30**

**5 Reading the Tutorial 3 figures** **38**

**6 Answers to the "Over to you" boxes** **41**
Slide 8: Shapes in the chain rule, and the order . . . 41
Slide 14: Monomials against Legendre on the same space . . . 41
Slide 17: Where did the derivation use one dimension? . . . 41
Slide 24: Measuring the decay rate without the exact solution . . . 41

Slide 45: Propose a rule that turns the residual into $n_b$ equations . . . . . 42
Slide 48: What makes $\boldsymbol{K}$ diagonal as well . . . . . 42
Slide 55: Petrov–Galerkin and the loss of symmetry . . . . . 42
Slide 67: $A^e_{11}$, and why $\boldsymbol{K}^e$ is singular . . . . . 43
Slide 72: The diffusive step limit, in numbers . . . . . 43
Slide 73: Why the nonlinear term is not treated implicitly too . . . . . 43
Slide 87: Which two-point quantity appears . . . . . 44
Slide 89: What $\boldsymbol{D}$ looks like for a steady solution . . . . . 44
Slide 95: What $\sum_{r>n_r} \sigma_r^2$ tells you before any model . . . . . 44
Slide 101: Which Tutorial 2 operators survive the change of basis . . . . . 45
Slide 106: The two quantities that fill the diagnosis table . . . . . 45
Slide 115: Is the union of two orthonormal sets orthonormal? . . . . . 45
Slide 122: Which stored object breaks if $x_f$ becomes a parameter . . . . . 45

**7 Answers to the tutorial questions** **47**

**8 Two derivations worth keeping in reserve** **50**

## 1 Section 2: approximation, inner products, projection

**Slide 10–11** Normal equations, in vector form

*We derive this once, in matrix notation, because the same three lines reappear for every method in the course.*

**Step 1. Name the objects.** Collect the basis in a matrix and the coefficients in a vector,

$$\boldsymbol{\Phi} = [\boldsymbol{\phi}_1 \ \boldsymbol{\phi}_2 \ \cdots \ \boldsymbol{\phi}_{n_b}] \in \mathbb{R}^{n_x \times n_b}, \qquad \boldsymbol{c} \in \mathbb{R}^{n_b}, \qquad \tilde{\boldsymbol{u}} = \boldsymbol{\Phi}\boldsymbol{c} \in \mathbb{R}^{n_x}.$$

*Ask the room for the shapes before writing them. $\boldsymbol{\Phi}$ is tall and thin: many samples, few basis functions. That shape is the reason the problem has no exact solution and must be projected.*

**Step 2. The error and the cost.**

$$\boldsymbol{e}(\boldsymbol{c}) = \boldsymbol{u} - \boldsymbol{\Phi}\boldsymbol{c}, \qquad J(\boldsymbol{c}) = \|\boldsymbol{e}\|^2 = \boldsymbol{e}^\top \boldsymbol{e}.$$

**Step 3. The chain rule, with the shapes written underneath.** $J$ is a scalar function of the vector $\boldsymbol{e}$, and $\boldsymbol{e}$ is a vector function of $\boldsymbol{c}$:

$$\underbrace{\frac{\partial J}{\partial \boldsymbol{c}}}_{1\times n_b} = \underbrace{\frac{\partial J}{\partial \boldsymbol{e}}}_{1\times n_x} \underbrace{\frac{\partial \boldsymbol{e}}{\partial \boldsymbol{c}}}_{n_x \times n_b}$$

*The shapes force the order of the product. Write them under each factor and the rest of the derivation cannot go wrong.*

The two Jacobians are standard:

$$\frac{\partial}{\partial \boldsymbol{e}}(\boldsymbol{e}^\top \boldsymbol{e}) = 2\boldsymbol{e}^\top, \qquad \frac{\partial}{\partial \boldsymbol{c}}(\boldsymbol{u} - \boldsymbol{\Phi}\boldsymbol{c}) = -\boldsymbol{\Phi}.$$

*If the first one is unfamiliar, expand $\boldsymbol{e}^\top \boldsymbol{e} = \sum_j e_j^2$ and differentiate with respect to $e_i$: only one term survives, giving $2e_i$. That is the $i$-th entry of the row vector $2\boldsymbol{e}^\top$.*

**Step 4. Set the gradient to zero.**

$$\frac{\partial J}{\partial \boldsymbol{c}} = (2\boldsymbol{e}^\top)(-\boldsymbol{\Phi}) = -2\,\boldsymbol{e}^\top \boldsymbol{\Phi} = \boldsymbol{0}^\top \qquad \Longleftrightarrow \qquad \boxed{\boldsymbol{\Phi}^\top \boldsymbol{e} = \boldsymbol{0}}$$

**Step 5. Read the boxed equation row by row.** Row $m$ of $\boldsymbol{\Phi}^\top \boldsymbol{e}$ is $\boldsymbol{\phi}_m^\top \boldsymbol{e} = \langle \boldsymbol{\phi}_m, \boldsymbol{e} \rangle$, so the single matrix statement is the $n_b$ scalar conditions

$$\langle \boldsymbol{\phi}_m, \boldsymbol{e} \rangle = 0, \qquad m = 1, \ldots, n_b.$$

*So the calculus and the geometry meet here: minimising the length of the error is the same as making it orthogonal to every basis vector.*

**Step 6. Substitute the error and obtain the normal equations.**

$$\boldsymbol{\Phi}^\top (\boldsymbol{u} - \boldsymbol{\Phi}\boldsymbol{c}) = \boldsymbol{0} \qquad \Longrightarrow \qquad \underbrace{\boldsymbol{\Phi}^\top \boldsymbol{\Phi}}_{\boldsymbol{A}}\, \boldsymbol{c} = \underbrace{\boldsymbol{\Phi}^\top \boldsymbol{u}}_{\boldsymbol{b}}$$

with $A_{mn} = \langle \boldsymbol{\phi}_m, \boldsymbol{\phi}_n \rangle$ and $b_m = \langle \boldsymbol{\phi}_m, \boldsymbol{u} \rangle$.

**Step 7. The two special cases.** An orthogonal basis makes $\boldsymbol{A}$ diagonal and the system falls apart into $n_b$ independent scalar equations, $c_n = \langle \boldsymbol{\phi}_n, \boldsymbol{u} \rangle \,/\, \|\boldsymbol{\phi}_n\|^2$. If in addition $\|\boldsymbol{\phi}_n\| = 1$, then $\boldsymbol{A} = \boldsymbol{I}$ and $c_n = \langle \boldsymbol{\phi}_n, \boldsymbol{u} \rangle$.

**With a weighted inner product.** Nothing changes except that $\boldsymbol{\Phi}^\top$ becomes $\boldsymbol{\Phi}^\top \boldsymbol{W}_s$:

$$J(\boldsymbol{c}) = \boldsymbol{e}^\top \boldsymbol{W}_s \boldsymbol{e} \quad \Longrightarrow \quad \frac{\partial J}{\partial \boldsymbol{c}} = -2\,\boldsymbol{e}^\top \boldsymbol{W}_s \boldsymbol{\Phi} \quad \Longrightarrow \quad \boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{\Phi}\, \boldsymbol{c} = \boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{u}.$$

This single line covers quadrature weights, the finite-element mass matrix, and the weighted POD of Section 4. Worth deriving once and pointing back to.

A numerical illustration that takes two minutes. Take $\boldsymbol{\phi}_1 = (1, 0, 0)^\top$ and $\boldsymbol{\phi}_2 = (1, \varepsilon, 0)^\top$ with $\varepsilon = 10^{-3}$, so the two vectors are almost parallel. Then

$$\boldsymbol{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1+\varepsilon^2 \end{bmatrix}, \qquad \det \boldsymbol{A} = \varepsilon^2 = 10^{-6}, \qquad \operatorname{cond}(\boldsymbol{A}) \approx 4 \cdot 10^6.$$

The plane spanned by the two vectors is perfectly healthy. It is the basis that is bad, and $\boldsymbol{A}$ is what reports it.

**Slide 13** Solving the normal equations with Cholesky

*We have a symmetric positive definite system. Never hand that to a general solver.*

**Step 1. Establish the two properties, on the board.**

$$A_{mn} = \langle \boldsymbol{\phi}_m, \boldsymbol{\phi}_n \rangle = \langle \boldsymbol{\phi}_n, \boldsymbol{\phi}_m \rangle = A_{nm} \qquad \text{(symmetry, from the inner product)}$$

$$\boldsymbol{z}^\top \boldsymbol{A} \boldsymbol{z} = \sum_{m,n} z_m \langle \boldsymbol{\phi}_m, \boldsymbol{\phi}_n \rangle z_n = \Big\langle \sum_m z_m \boldsymbol{\phi}_m,\ \sum_n z_n \boldsymbol{\phi}_n \Big\rangle = \Big\| \sum_n z_n \boldsymbol{\phi}_n \Big\|^2 \ > \ 0$$

for $\boldsymbol{z} \neq \boldsymbol{0}$, *provided* the $\boldsymbol{\phi}_n$ are independent. *Read the last line backwards: positive definiteness of $\boldsymbol{A}$ is exactly the statement that the basis is linearly independent. The algebra and the geometry are the same fact.*

**Step 2. Factor, with $n_b = 3$, and watch where each entry comes from.** Write $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^\top$ with

$$\boldsymbol{L} = \begin{bmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{bmatrix}$$

and multiply out, entry by entry, comparing with $\boldsymbol{A}$:

$$\begin{aligned}
A_{11} &= \ell_{11}^2 && \Rightarrow \quad \ell_{11} = \sqrt{A_{11}} \\
A_{21} &= \ell_{21}\ell_{11} && \Rightarrow \quad \ell_{21} = A_{21}/\ell_{11} \\
A_{31} &= \ell_{31}\ell_{11} && \Rightarrow \quad \ell_{31} = A_{31}/\ell_{11} \\
A_{22} &= \ell_{21}^2 + \ell_{22}^2 && \Rightarrow \quad \ell_{22} = \sqrt{A_{22} - \ell_{21}^2} \\
A_{32} &= \ell_{31}\ell_{21} + \ell_{32}\ell_{22} && \Rightarrow \quad \ell_{32} = (A_{32} - \ell_{31}\ell_{21})/\ell_{22} \\
A_{33} &= \ell_{31}^2 + \ell_{32}^2 + \ell_{33}^2 && \Rightarrow \quad \ell_{33} = \sqrt{A_{33} - \ell_{31}^2 - \ell_{32}^2}
\end{aligned}$$

*Each unknown is determined the moment it first appears, in this order. No choices, no pivoting, no search: that is why Cholesky is both cheap and stable.*

**Step 3. Solve in two triangular sweeps.**

$$\boldsymbol{A}\boldsymbol{c} = \boldsymbol{b} \iff \boldsymbol{L}\underbrace{\boldsymbol{L}^\top \boldsymbol{c}}_{\boldsymbol{y}} = \boldsymbol{b} \qquad \Rightarrow \qquad \boldsymbol{L}\boldsymbol{y} = \boldsymbol{b} \text{ (forward)}, \quad \boldsymbol{L}^\top \boldsymbol{c} = \boldsymbol{y} \text{ (backward)}.$$

Write out the forward sweep for $n_b = 3$, so that the substitution is visible:

$$y_1 = \frac{b_1}{\ell_{11}}, \qquad y_2 = \frac{b_2 - \ell_{21} y_1}{\ell_{22}}, \qquad y_3 = \frac{b_3 - \ell_{31} y_1 - \ell_{32} y_2}{\ell_{33}}.$$

*Each unknown needs only the ones already computed, so the sweep costs $\mathcal{O}(n_b^2)$ operations, not $\mathcal{O}(n_b^3)$. In code this means* `solve_triangular`*, not a general solver: handing a triangular matrix to a general solver runs a full LU on it and throws away the structure that Cholesky just produced.*

**Step 4. Count, and note the failure mode.**

| | cost | needs pivoting? |
|---|---|---|
| *LU* on a general matrix | $\frac{2}{3} n_b^3$ | yes |
| Cholesky on an SPD matrix | $\frac{1}{3} n_b^3$ | no |

The square roots are the diagnostic: the factorisation fails, by asking for the square root of a negative number, exactly when $\boldsymbol{A}$ stops being numerically positive definite. That is the machine telling you the basis has gone dependent. *So when the code raises an exception, do not switch to a more forgiving solver. Change the basis.*

Cholesky and Gram–Schmidt are the same factorisation from two sides. Let $\boldsymbol{W} = [\boldsymbol{w}_1, \ldots, \boldsymbol{w}_{n_b}]$ hold the original vectors, so that $\boldsymbol{A} = \boldsymbol{W}^\top \boldsymbol{W}$. Gram–Schmidt gives $\boldsymbol{W} = \boldsymbol{\Phi} \boldsymbol{R}$ with $\boldsymbol{R}$ upper triangular, and therefore

$$\boldsymbol{A} = \boldsymbol{W}^\top \boldsymbol{W} = \boldsymbol{R}^\top \boldsymbol{\Phi}^\top \boldsymbol{\Phi} \boldsymbol{R} = \boldsymbol{R}^\top \boldsymbol{R},$$

which is the Cholesky factorisation with $\boldsymbol{L} = \boldsymbol{R}^\top$. Orthogonalising the basis and factoring the Gram matrix are two ways of doing the same work.

### Slide 12 Gram–Schmidt

Take three linearly independent vectors $\boldsymbol{w}_1, \boldsymbol{w}_2, \boldsymbol{w}_3$ and build an orthonormal basis of the same space, one vector at a time.

**First vector.** Nothing to remove:

$$\tilde{\boldsymbol{\phi}}_1 = \boldsymbol{w}_1, \qquad \boldsymbol{\phi}_1 = \frac{\tilde{\boldsymbol{\phi}}_1}{\left\| \tilde{\boldsymbol{\phi}}_1 \right\|}.$$

**Second vector.** Remove from $\boldsymbol{w}_2$ the part that $\boldsymbol{\phi}_1$ can already represent:

$$\tilde{\boldsymbol{\phi}}_2 = \boldsymbol{w}_2 - \underbrace{\langle \boldsymbol{\phi}_1, \boldsymbol{w}_2 \rangle \, \boldsymbol{\phi}_1}_{\text{the projection of } \boldsymbol{w}_2 \text{ on } \boldsymbol{\phi}_1}, \qquad \boldsymbol{\phi}_2 = \frac{\tilde{\boldsymbol{\phi}}_2}{\left\| \tilde{\boldsymbol{\phi}}_2 \right\|}.$$

Check on the board that it worked:

$$\left\langle \boldsymbol{\phi}_1, \tilde{\boldsymbol{\phi}}_2 \right\rangle = \langle \boldsymbol{\phi}_1, \boldsymbol{w}_2 \rangle - \langle \boldsymbol{\phi}_1, \boldsymbol{w}_2 \rangle \underbrace{\langle \boldsymbol{\phi}_1, \boldsymbol{\phi}_1 \rangle}_{=1} = 0.$$

**Third vector.** Remove what *both* previous directions can represent:

$$\tilde{\boldsymbol{\phi}}_3 = \boldsymbol{w}_3 - \langle \boldsymbol{\phi}_1, \boldsymbol{w}_3 \rangle \, \boldsymbol{\phi}_1 - \langle \boldsymbol{\phi}_2, \boldsymbol{w}_3 \rangle \, \boldsymbol{\phi}_2, \qquad \boldsymbol{\phi}_3 = \frac{\tilde{\boldsymbol{\phi}}_3}{\left\| \tilde{\boldsymbol{\phi}}_3 \right\|}.$$

*At every step we subtract everything already representable and keep what is new. That is the whole algorithm.*

**What it is, in one line.** Invert the three relations for $\boldsymbol{w}_1, \boldsymbol{w}_2, \boldsymbol{w}_3$:

$$\underbrace{[\boldsymbol{w}_1\ \boldsymbol{w}_2\ \boldsymbol{w}_3]}_{\boldsymbol{W}} = \underbrace{[\boldsymbol{\phi}_1\ \boldsymbol{\phi}_2\ \boldsymbol{\phi}_3]}_{\boldsymbol{\Phi}} \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix}}_{\boldsymbol{R}}, \qquad r_{mn} = \langle \boldsymbol{\phi}_m, \boldsymbol{w}_n \rangle\,, \quad r_{nn} = \left\| \tilde{\boldsymbol{\phi}}_n \right\|.$$

$\boldsymbol{R}$ is upper triangular precisely because $\boldsymbol{\phi}_m$ was built from $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_m$ only. This is the QR factorisation, and $\boldsymbol{A} = \boldsymbol{W}^\top \boldsymbol{W} = \boldsymbol{R}^\top \boldsymbol{R}$ is its Cholesky factorisation.

Worth stating explicitly: Gram–Schmidt changes the *basis*, never the *space*. The approximation $\tilde{\boldsymbol{u}}$ is exactly the same before and after; only the numbers $c_n$ and the conditioning change. Students often expect the accuracy to improve.

**Slide 16** Projection in function space

*Everything on slide 10 survives word for word. Only the inner product changes.*

Take $\Omega = (0, 1)$, $\phi_n(x) = \sqrt{2}\sin(n\pi x)$ and $n_b = 3$.

**Step 1. The Gram matrix, computed by hand.** Use $2\sin a \sin b = \cos(a-b) - \cos(a+b)$:

$$A_{mn} = \int_0^1 2\sin(m\pi x)\sin(n\pi x)\,\mathrm{d}x = \int_0^1 \left[\cos((m-n)\pi x) - \cos((m+n)\pi x)\right] \mathrm{d}x.$$

For $m \neq n$ both integrals vanish, since $\int_0^1 \cos(k\pi x)\,\mathrm{d}x = 0$ for any non-zero integer $k$. For $m = n$ the first integrand is 1 and the second still integrates to zero. Hence

$$A_{mn} = \delta_{mn}, \qquad \boldsymbol{A} = \boldsymbol{I}_3.$$

**Step 2. The coefficients then come for free.**

$$c_n = \langle f, \phi_n \rangle_\Omega = \int_0^1 f(x)\,\sqrt{2}\sin(n\pi x)\,\mathrm{d}x.$$

**Step 3. A worked example.** Take $f(x) = x(1-x)$. Two integrations by parts give

$$\int_0^1 x(1-x)\sin(n\pi x)\,\mathrm{d}x = \frac{2(1-(-1)^n)}{n^3\pi^3}, \qquad \text{so} \qquad c_n = \frac{4\sqrt{2}}{n^3\pi^3}\ \ (n \text{ odd}), \qquad c_n = 0\ \ (n \text{ even}).$$

Numerically $c_1 = 0.18244$, $c_2 = 0$, $c_3 = 0.0067571$. *Ask why every even coefficient vanishes before computing them: $f$ is symmetric about $x = 1/2$, and $\sin(2\pi x)$, $\sin(4\pi x)$ are antisymmetric about that point, so those integrals cancel by symmetry alone.*

**Step 4. Read the error off Pythagoras.** Since the basis is orthonormal,

$$\|f\|_{L^2}^2 = \sum_{n \geq 1} c_n^2 \qquad \Longrightarrow \qquad \|f - f_{n_b}\|_{L^2}^2 = \|f\|_{L^2}^2 - \sum_{n=1}^{n_b} c_n^2.$$

With $\|f\|_{L^2}^2 = \int_0^1 x^2(1-x)^2\,\mathrm{d}x = 1/30$:

| $n_b$ | $\sum_{n \leq n_b} c_n^2$ | $\|f - f_{n_b}\|_{L^2}$ | relative |
|---|---|---|---|
| 1 | 0.0332852 | $6.94 \cdot 10^{-3}$ | $3.80 \cdot 10^{-2}$ |
| 3 | 0.0333308 | $1.58 \cdot 10^{-3}$ | $8.67 \cdot 10^{-3}$ |
| 5 | 0.0333330 | $6.14 \cdot 10^{-4}$ | $3.36 \cdot 10^{-3}$ |
| 7 | 0.0333332 | $3.07 \cdot 10^{-4}$ | $1.68 \cdot 10^{-3}$ |

This example is a good trap to spring deliberately. $f$ is a polynomial, so students expect spectral accuracy, and the error instead falls like $n_b^{-3/2}$. The reason is that a sine series represents the *odd periodic extension* of $f$, and that extension has a jump in its second derivative at $x = 0$. Smoothness of $f$ on $\Omega$ is not enough: what matters is the smoothness of the function the basis actually reconstructs. This is the honest version of the "convergence depends on smoothness" slide.

**Slide 18** Discrete inner products: from integrals to matrices

**Step 1. Replace the integral by a quadrature.** On $n_x$ points with weights $w_j$ (trapezoid, midpoint, Gauss, whatever),

$$\langle a, b\rangle_\Omega = \int_\Omega a\, b \,\mathrm{d}\Omega \;\approx\; \sum_{j=1}^{n_x} w_j\, a(x_j)\, b(x_j) \;=\; \boldsymbol{b}^\top \boldsymbol{W}_s\, \boldsymbol{a}, \qquad \boldsymbol{W}_s = \mathrm{diag}(w_1, \ldots, w_{n_x}).$$

**Step 2. Sample the basis.** With $n_b = 3$ and $n_x = 5$,

$$\boldsymbol{\Phi}(\boldsymbol{x}_*) = \begin{bmatrix} \phi_1(x_1) & \phi_2(x_1) & \phi_3(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \phi_3(x_2) \\ \phi_1(x_3) & \phi_2(x_3) & \phi_3(x_3) \\ \phi_1(x_4) & \phi_2(x_4) & \phi_3(x_4) \\ \phi_1(x_5) & \phi_2(x_5) & \phi_3(x_5) \end{bmatrix} \in \mathbb{R}^{5\times 3}, \qquad \boldsymbol{f}_* = \begin{bmatrix} f(x_1) \\ \vdots \\ f(x_5) \end{bmatrix} \in \mathbb{R}^5.$$

*Columns are basis functions, rows are grid points. Five equations, three unknowns: the system $\boldsymbol{f}_* = \boldsymbol{\Phi c}$ is overdetermined, and that is exactly why we project instead of interpolating.*

**Step 3. Project.** Multiply by $\boldsymbol{\Phi}^\top \boldsymbol{W}_s$:

$$\underbrace{\boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{\Phi}}_{\boldsymbol{A} \,\in\, \mathbb{R}^{3\times 3}} \boldsymbol{c} = \underbrace{\boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{f}_*}_{\boldsymbol{b} \,\in\, \mathbb{R}^3}$$

and write the first row out in full so the pattern is visible:

$$\sum_{j=1}^{5} w_j \phi_1(x_j)^2\, c_1 + \sum_{j=1}^{5} w_j \phi_1(x_j)\phi_2(x_j)\, c_2 + \sum_{j=1}^{5} w_j \phi_1(x_j)\phi_3(x_j)\, c_3 = \sum_{j=1}^{5} w_j \phi_1(x_j) f(x_j).$$

**Step 4. Two things can now go wrong, and they are different.**

- the basis is not orthogonal, so $\boldsymbol{A}$ is full: a modelling choice;
- the basis is orthogonal but the quadrature is too coarse to see it, so $\boldsymbol{A}$ is only approximately diagonal: a numerical accident.

*The second is the one that bites in the tutorial. With $n_b$ sine modes you need enough points to integrate $\sin(2n_b\pi x)$ accurately, not just $\sin(n_b\pi x)$, because the products of basis functions oscillate twice as fast as the basis functions themselves.*

**Slide 21** Two errors, and only one is our fault

Let $P$ be the orthogonal projector onto $\mathcal{V}$, so $Pu \in \mathcal{V}$ and $u - Pu \perp \mathcal{V}$. Let $u_h \in \mathcal{V}$ be whatever our method actually produced.

**Step 1. Add and subtract.**

$$u - u_h = (u - Pu) + (Pu - u_h)$$

**Step 2. Note where each piece lives.** $Pu - u_h$ is a difference of two elements of $\mathcal{V}$, so it lies *in* $\mathcal{V}$; and $u - Pu$ is orthogonal to everything in $\mathcal{V}$. Therefore the two pieces are orthogonal to each other.

**Step 3. Pythagoras.**

$$\|u - u_h\|_{L^2}^2 = \|u - Pu\|_{L^2}^2 + \|Pu - u_h\|_{L^2}^2$$

Three consequences worth writing next to it.

1. $\|u - u_h\|_{L^2} \geq \|u - Pu\|_{L^2}$ always: *no method can beat the space it works in.*
2. The two terms are separately measurable in a numerical experiment. The first needs the exact solution projected; the second compares the computed coefficients against the projected ones.
3. If the first dominates, changing the scheme is wasted effort: enrich the space. If the second dominates, enriching the space is wasted effort: fix the scheme.

*We will run exactly this diagnostic on the reduced model in Tutorial 3, where the first term is* $\varepsilon_{\text{proj}}$ *and the total is* $e_{\text{ROM}}$.

## 2 Section 3: Galerkin projection of a PDE

**Slide 29** The Galerkin condition

*We cannot use slide 10 directly, because there we needed $\langle \phi_m, u \rangle_\Omega$ and $u$ is unknown. Here is the substitution that saves us.*

**Step 1. What we would like, and cannot have.**

$$\langle \phi_m,\, u - u_{n_b} \rangle_\Omega = 0 \qquad \text{requires knowing } u.$$

**Step 2. What we do know.** $u$ satisfies $\partial_t u = \mathcal{F}(u)$. Define the residual by feeding our approximation into the same equation:

$$\mathcal{R}(x,t) = \partial_t u_{n_b} - \mathcal{F}(u_{n_b}; \boldsymbol{\mu}).$$

Every ingredient is computable, because $u_{n_b}$ is known once $\boldsymbol{a}(t)$ is.

**Step 3. Impose orthogonality on the residual instead of on the error.**

$$\langle \mathcal{R}, \phi_m \rangle_\Omega = 0, \qquad m = 1, \ldots, n_b.$$

$n_b$ unknowns, $n_b$ equations.

Ask the obvious question before a student does: *why should making the residual small make the error small?* Answer honestly. In general it is an assumption. For a symmetric coercive operator it is a theorem, Cea's lemma, which says

$$\|u - u_{n_b}\|_a \leq \inf_{w \in \mathcal{V}} \|u - w\|_a \,,$$

that is, the Galerkin solution is the best approximation in the energy norm. Away from that setting, a small residual guarantees a small error only up to the conditioning of the operator, which is exactly why convection-dominated problems misbehave.

**Slide 29b** The Galerkin condition in the continuous setting

*So far the residual condition was stated. Here is where it comes from, in function space, without ever discretising.*

**Step 1. The problem in weak form.** Let $\mathcal{L}$ be a linear operator and look for $u$ in a space $\mathcal{V}$ with

$$\mathcal{L}u = f \qquad \text{in } \Omega.$$

Multiply by an arbitrary test function $\psi \in \mathcal{V}$ and integrate:

$$\langle \mathcal{L}u, \psi \rangle_\Omega = \langle f, \psi \rangle_\Omega \qquad \text{for every } \psi \in \mathcal{V}.$$

*Nothing has been lost. If this holds for* every *$\psi$, and the functions involved are continuous, then $\mathcal{L}u - f$ vanishes pointwise. That implication is the fundamental lemma of the calculus of variations, and it is what makes the weak form equivalent to the strong one.*

**Step 2. Restrict to a finite subspace.** Replace $\mathcal{V}$ by $\mathcal{V}_{n_b} = \operatorname{span}\{\phi_1, \ldots, \phi_{n_b}\} \subset \mathcal{V}$ on both sides: seek $u_{n_b} \in \mathcal{V}_{n_b}$ with

$$\langle \mathcal{L}u_{n_b}, \psi \rangle_\Omega = \langle f, \psi \rangle_\Omega \qquad \text{for every } \psi \in \mathcal{V}_{n_b}.$$

*Two restrictions, not one. The solution is restricted, and so is the set of functions we test against. Both matter.*

**Step 3. Test functions may be taken one at a time.** Any $\psi \in \mathcal{V}_{n_b}$ is $\psi = \sum_m z_m \phi_m$, so

$$\langle \mathcal{L}u_{n_b} - f, \psi \rangle_\Omega = \sum_{m=1}^{n_b} z_m \, \langle \mathcal{L}u_{n_b} - f, \phi_m \rangle_\Omega \, .$$

Requiring this to vanish for *all* coefficients $z_m$ forces each bracket to vanish separately:

$$\boxed{\langle \mathcal{R}, \phi_m \rangle_\Omega = 0, \qquad m = 1, \ldots, n_b, \qquad \mathcal{R} = \mathcal{L}u_{n_b} - f}$$

*This is the step worth slowing down on. "Orthogonal to the whole space" and "orthogonal to each of the $n_b$ basis functions" are the same statement, because the inner product is linear. That is the only reason we may write $n_b$ separate equations.*

**Step 4. The geometric reading.** $\mathcal{R}$ is a function. The condition says

$$\mathcal{R} \perp \mathcal{V}_{n_b}, \qquad \text{that is} \qquad P_{\mathcal{V}_{n_b}} \mathcal{R} = 0,$$

so the residual has *no component at all* inside the space where we are looking for the solution. It is entirely made of the directions we cannot see.

**The parallel to put on the board, side by side.**

| | Section 2, known $u$ | Section 3, unknown $u$ |
|---|---|---|
| object made orthogonal | the error $e = u - u_{n_b}$ | the residual $\mathcal{R}$ |
| condition | $\langle \phi_m, e \rangle_\Omega = 0$ | $\langle \phi_m, \mathcal{R} \rangle_\Omega = 0$ |
| computable? | only if $u$ is known | always |
| gives | the best approximation | the Galerkin solution |

The two coincide when $\mathcal{L}$ is symmetric and coercive, which is Cea's lemma at the end of these notes. Otherwise the Galerkin solution is merely good, not optimal.

**Slide 30–31** From the PDE to a system of ODEs

Take the linear problem $\partial_t u = \mathcal{L}u + f$ and $n_b = 3$.

**Step 1. The ansatz, written out.**

$$u_3(x,t) = a_1(t)\phi_1(x) + a_2(t)\phi_2(x) + a_3(t)\phi_3(x)$$

**Step 2. The two derivatives.** Time hits only the coefficients, space hits only the basis:

$$\partial_t u_3 = \dot{a}_1\phi_1 + \dot{a}_2\phi_2 + \dot{a}_3\phi_3, \qquad \mathcal{L}u_3 = a_1\mathcal{L}\phi_1 + a_2\mathcal{L}\phi_2 + a_3\mathcal{L}\phi_3.$$

*This is the only place where "the basis does not depend on time" is used, and it is what makes the whole method work. If the basis moved, we would pick up extra terms here. Remember this when we discuss adaptive bases.*

**Step 3. The residual, in full.**

$$\mathcal{R} = \dot{a}_1\phi_1 + \dot{a}_2\phi_2 + \dot{a}_3\phi_3 - a_1\mathcal{L}\phi_1 - a_2\mathcal{L}\phi_2 - a_3\mathcal{L}\phi_3 - f$$

**Step 4. Three equations, written out one by one.** Test with $\phi_1$:

$$\dot{a}_1 \langle \phi_1, \phi_1 \rangle_\Omega + \dot{a}_2 \langle \phi_1, \phi_2 \rangle_\Omega + \dot{a}_3 \langle \phi_1, \phi_3 \rangle_\Omega = a_1 \langle \phi_1, \mathcal{L}\phi_1 \rangle_\Omega + a_2 \langle \phi_1, \mathcal{L}\phi_2 \rangle_\Omega + a_3 \langle \phi_1, \mathcal{L}\phi_3 \rangle_\Omega + \langle \phi_1, f \rangle_\Omega$$

Test with $\phi_2$:

$$\dot{a}_1 \langle \phi_2, \phi_1 \rangle_\Omega + \dot{a}_2 \langle \phi_2, \phi_2 \rangle_\Omega + \dot{a}_3 \langle \phi_2, \phi_3 \rangle_\Omega = a_1 \langle \phi_2, \mathcal{L}\phi_1 \rangle_\Omega + a_2 \langle \phi_2, \mathcal{L}\phi_2 \rangle_\Omega + a_3 \langle \phi_2, \mathcal{L}\phi_3 \rangle_\Omega + \langle \phi_2, f \rangle_\Omega$$

Test with $\phi_3$: same pattern with the index 3.

**Step 5. Collapse.**

$$\underbrace{\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix}}_{\boldsymbol{A}} \begin{bmatrix} \dot{a}_1 \\ \dot{a}_2 \\ \dot{a}_3 \end{bmatrix} = \underbrace{\begin{bmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{bmatrix}}_{\boldsymbol{K}} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix}$$

with $A_{mn} = \langle \phi_m, \phi_n \rangle_\Omega$, $K_{mn} = \langle \phi_m, \mathcal{L}\phi_n \rangle_\Omega$ and $f_m = \langle \phi_m, f \rangle_\Omega$.

**The three questions to ask the room at this point.**

1. What is $\boldsymbol{A}$ if the basis is orthonormal? *The identity, and the system becomes explicit.*
2. What is $\boldsymbol{K}$ if the $\phi_n$ are eigenfunctions of $\mathcal{L}$? *Diagonal, and the three equations decouple completely.*
3. What have we gained over finite differences? *Nothing yet. The gain appears only when $n_b \ll n_x$, which is what Section 4 is about.*

**Slide 32–33** The quadratic tensor, written out

Now let $\mathcal{F}$ contain a quadratic term. Use the convective term of the tutorial, $-v\,\partial_x v$, and keep $n_b = 3$.

**Step 1. The product of two sums is a double sum.**

$$v\,\partial_x v = (a_1\phi_1 + a_2\phi_2 + a_3\phi_3)(a_1\phi_1' + a_2\phi_2' + a_3\phi_3')$$

Multiply it out completely, all nine terms:

$$\begin{aligned} v\,\partial_x v = {} & a_1a_1\,\phi_1\phi_1' + a_1a_2\,\phi_1\phi_2' + a_1a_3\,\phi_1\phi_3' \\ & + a_2a_1\,\phi_2\phi_1' + a_2a_2\,\phi_2\phi_2' + a_2a_3\,\phi_2\phi_3' \\ & + a_3a_1\,\phi_3\phi_1' + a_3a_2\,\phi_3\phi_2' + a_3a_3\,\phi_3\phi_3' \end{aligned}$$

*Nine products of functions, each multiplied by a product of two unknowns. The unknowns are outside the functions, which is the whole point: they can be pulled out of the integral.*

**Step 2. Test with $\phi_1$ and pull the coefficients out.**

$$\begin{aligned} -\langle v\,\partial_x v, \phi_1 \rangle_\Omega = {} & -a_1a_1\,\langle \phi_1\phi_1', \phi_1 \rangle_\Omega - a_1a_2\,\langle \phi_1\phi_2', \phi_1 \rangle_\Omega - a_1a_3\,\langle \phi_1\phi_3', \phi_1 \rangle_\Omega \\ & - a_2a_1\,\langle \phi_2\phi_1', \phi_1 \rangle_\Omega - a_2a_2\,\langle \phi_2\phi_2', \phi_1 \rangle_\Omega - a_2a_3\,\langle \phi_2\phi_3', \phi_1 \rangle_\Omega \\ & - a_3a_1\,\langle \phi_3\phi_1', \phi_1 \rangle_\Omega - a_3a_2\,\langle \phi_3\phi_2', \phi_1 \rangle_\Omega - a_3a_3\,\langle \phi_3\phi_3', \phi_1 \rangle_\Omega \end{aligned}$$

**Step 3. Name the numbers.** Each bracket is a number, fixed once the basis is fixed. Call it

$$\boxed{C_{mnk} = -\int_\Omega \phi_m(x)\,\phi_n(x)\,\phi_k'(x)\,\mathrm{d}x}$$

so that the line above is

$$\left[\boldsymbol{C} : (\boldsymbol{a} \otimes \boldsymbol{a})\right]_1 = \sum_{n=1}^{3}\sum_{k=1}^{3} C_{1nk} a_n a_k.$$

Repeating for $m = 2$ and $m = 3$ gives three such lines, and $3 \times 3 \times 3 = 27$ numbers in total.

**Step 4. Read the index structure.**

| | |
|---|---|
| $m$ | which equation we are writing, that is which test function |
| $n$ | which basis function appears *undifferentiated* |
| $k$ | which basis function appears *differentiated* |

Because $n$ and $k$ play different roles, $\boldsymbol{C}$ is *not* symmetric in its last two indices: $C_{mnk} \neq C_{mkn}$ in general. But $a_n a_k = a_k a_n$, so only the combination $C_{mnk} + C_{mkn}$ ever affects the answer.

**Step 5. For the sine basis, all 27 numbers in closed form.** With $\phi_n = \sqrt{2}\sin(n\pi x)$ on $(0,1)$,

$$C_{mnk} = -2\sqrt{2}\,k\pi \int_0^1 \sin(m\pi x)\sin(n\pi x)\cos(k\pi x)\,\mathrm{d}x.$$

Use $2\sin a \sin b = \cos(a-b) - \cos(a+b)$ and then $\int_0^1 \cos(p\pi x)\cos(k\pi x)\,\mathrm{d}x = \frac{1}{2}\delta_{pk}$ for $p, k \geq 1$:

$$\boxed{C_{mnk} = -\frac{\sqrt{2}\,k\pi}{2}\left(\delta_{|m-n|,k} - \delta_{m+n,k}\right)}$$

*Two selection rules. A triad contributes only when the differentiated mode number equals either the difference or the sum of the other two. Everything else is exactly zero, which is why the tensor is sparse for this basis.*

Spot values, useful as a check on the students' code:

$$C_{112} = +\sqrt{2}\pi \approx 4.443, \qquad C_{121} = C_{211} = -\tfrac{\sqrt{2}\pi}{2} \approx -2.221, \qquad C_{123} = \tfrac{3\sqrt{2}\pi}{2} \approx 6.664.$$

Two errors that cost students an hour each.

1. Forgetting that the middle index is the undifferentiated one, which transposes $\boldsymbol{C}$ and produces a plausible but wrong solution.
2. Under-resolving the quadrature. The integrand $\phi_m\phi_n\phi_k'$ oscillates like $\sin(3n_b\pi x)$ in the worst case, so a grid that resolves the basis does not necessarily resolve the tensor.

**Slide 54** Self-adjoint $\mathcal{L}$ gives symmetric $\boldsymbol{K}$, and skew gives antisymmetric

*The slide asserts that Galerkin inherits the structure of the operator. This is the proof, and it is three lines. Then do the two examples, because the second one is the more instructive.*

### The general statement

$\mathcal{L}$ is *self-adjoint* on the space where the $\phi_n$ live when

$$\langle w, \mathcal{L}u\rangle_\Omega = \langle \mathcal{L}w, u\rangle_\Omega \qquad \text{for every admissible } u, w.$$

The projected operator is $K_{mn} = \langle \phi_m, \mathcal{L}\phi_n\rangle_\Omega$. Then

$$K_{mn} = \langle \phi_m, \mathcal{L}\phi_n\rangle_\Omega \overset{\text{self-adjoint}}{=} \langle \mathcal{L}\phi_m, \phi_n\rangle_\Omega \overset{\text{real inner product}}{=} \langle \phi_n, \mathcal{L}\phi_m\rangle_\Omega \;=\; K_{nm}.$$

$$\boxed{\mathcal{L} = \mathcal{L}^* \implies \boldsymbol{K} = \boldsymbol{K}^\top}$$

*Two facts were used and both are worth naming out loud. The first is the definition of self-adjointness. The second is only that a real inner product is symmetric, $\langle f, g\rangle_\Omega = \langle g, f\rangle_\Omega$. Nothing about the basis was assumed, so this holds for sines, for hats and for POD modes alike.*

The step from the first equality to the second is where Galerkin is *used*. It needs $\phi_m$ to be admissible as a *test* function, which is true precisely because the test space equals the trial space. Take $\psi_m \neq \phi_m$, as Petrov–Galerkin does, and $K_{mn} = \langle \psi_m, \mathcal{L}\phi_n\rangle_\Omega$ has no reason to equal $K_{nm}$. That single line is the whole difference between the two methods as far as structure is concerned.

**Example 1: diffusion is self-adjoint**

Take $\mathcal{L}u = \nu\, u''$ on $(0,1)$ with $u(0) = u(1) = 0$, and integrate by parts once:

$$\langle w, \mathcal{L}u\rangle_\Omega = \nu \int_0^1 w\, u''\,\mathrm{d}x = \nu \underbrace{\big[w\, u'\big]_0^1}_{=0 \text{ since } w(0)=w(1)=0} - \nu \int_0^1 w'\, u'\,\mathrm{d}x = -\nu \int_0^1 w' u'\,\mathrm{d}x.$$

The result is *manifestly* symmetric in $u$ and $w$: swapping them changes nothing. So $\langle w, \mathcal{L}u\rangle_\Omega = \langle \mathcal{L}w, u\rangle_\Omega$ and $\boldsymbol{K}$ is symmetric.

*Notice that the boundary term is what does the work. Self-adjointness is not a property of $\partial_{xx}$ alone; it is a property of $\partial_{xx}$* together with these boundary conditions. *Change to a Robin condition and the boundary term survives.*

With the sine basis, $\phi_n = \sqrt{2}\sin(n\pi x)$ and $n_b = 3$:

$$K_{mn} = -\nu \int_0^1 \phi_m' \phi_n'\,\mathrm{d}x = -\nu (n\pi)^2 \delta_{mn} \qquad \Longrightarrow \qquad \boldsymbol{K} = -\nu\pi^2 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 9 \end{bmatrix},$$

symmetric, and in fact diagonal because these $\phi_n$ are eigenfunctions.

**Example 2: convection is skew-adjoint**

Now take $\mathcal{L}u = -a\, u'$ with $a$ constant, same boundary conditions:

$$\langle w, \mathcal{L}u\rangle_\Omega = -a \int_0^1 w\, u'\,\mathrm{d}x = -a \underbrace{\big[w\, u\big]_0^1}_{=0} + a \int_0^1 w'\, u\,\mathrm{d}x = \langle -\mathcal{L}w, u\rangle_\Omega .$$

So $\mathcal{L}^* = -\mathcal{L}$: the operator is skew-adjoint, and the same three lines as before now give

$$\boxed{\mathcal{L}^* = -\mathcal{L} \implies \boldsymbol{K} = -\boldsymbol{K}^\top}$$

With the same three sines and $a = 1$, $B_{mn} = -\langle \phi_m, \phi_n'\rangle_\Omega$ evaluates in closed form to

$$B_{mn} = -\frac{4mn}{m^2 - n^2} \quad (m+n \text{ odd}), \qquad B_{mn} = 0 \quad (m+n \text{ even}),$$

$$\boldsymbol{B} = \begin{bmatrix} 0 & \frac{8}{3} & 0 \\ -\frac{8}{3} & 0 & \frac{24}{5} \\ 0 & -\frac{24}{5} & 0 \end{bmatrix}, \qquad \boldsymbol{B} = -\boldsymbol{B}^\top.$$

*Have them check one entry. $B_{12} = -4 \cdot 1 \cdot 2/(1-4) = 8/3$ and $B_{21} = -4 \cdot 2 \cdot 1/(4-1) = -8/3$. The zeros on $B_{13}$ come from $m+n = 4$ being even.*

**Why the antisymmetry matters: the energy**

Multiply the projected system by $\boldsymbol{a}^\top$ and look at the convective contribution alone:

$$\frac{\mathrm{d}}{\mathrm{d}t}\frac{\|\boldsymbol{a}\|^2}{2} = \boldsymbol{a}^\top \dot{\boldsymbol{a}} \quad \supset \quad \boldsymbol{a}^\top \boldsymbol{B}\, \boldsymbol{a}.$$

For any antisymmetric $\boldsymbol{B}$, $\boldsymbol{a}^\top \boldsymbol{B}\boldsymbol{a}$ is a scalar equal to its own transpose, so

$$\boldsymbol{a}^\top \boldsymbol{B}\boldsymbol{a} = (\boldsymbol{a}^\top \boldsymbol{B}\boldsymbol{a})^\top = \boldsymbol{a}^\top \boldsymbol{B}^\top \boldsymbol{a} = -\boldsymbol{a}^\top \boldsymbol{B}\boldsymbol{a} \qquad \Longrightarrow \qquad \boldsymbol{a}^\top \boldsymbol{B}\boldsymbol{a} = 0.$$

**The point of the whole slide, in one sentence.** Convection transports energy between modes and creates none. Diffusion removes it. A Galerkin projection keeps both statements exactly true at the discrete level, because it keeps $\boldsymbol{K}$ symmetric negative and $\boldsymbol{B}$ antisymmetric. A method that breaks the antisymmetry can manufacture energy out of the convective term, and that is one of the ways a reduced model blows up in Section 4.

The argument above is for the *linear* convection by a fixed field. The true Burgers term is quadratic and the corresponding statement involves the tensor: energy conservation needs $C_{mnk}+C_{nkm}+C_{kmn}=0$, the cyclic sum, rather than a two-index antisymmetry. Truncating a basis does not preserve that sum in general, which is exactly why truncated Galerkin models of nonlinear equations can drift.

**Slide 50–51** Petrov–Galerkin, and why it stabilises

*This is the part that usually stays a slogan. Here is the mechanism.*

### Where the trouble comes from

Take steady convection and diffusion, $a\,\partial_x u = \nu\,\partial_{xx} u$. Project on a basis and split the operator matrix in two:

$$\boldsymbol{K} = \underbrace{\boldsymbol{K}^{\mathrm{conv}}}_{\text{skew-symmetric}} + \underbrace{\boldsymbol{K}^{\mathrm{diff}}}_{\text{symmetric negative}}$$

The convective block is skew-symmetric, $\boldsymbol{K}^{\mathrm{conv}\top} = -\boldsymbol{K}^{\mathrm{conv}}$, whenever the test and trial spaces coincide and the boundary terms vanish. Show why on the board, by parts:

$$\int_0^1 \phi_m\, a\, \phi_n'\,\mathrm{d}x = \big[a\,\phi_m\phi_n\big]_0^1 - \int_0^1 a\,\phi_m'\phi_n\,\mathrm{d}x = -\int_0^1 \phi_n\, a\,\phi_m'\,\mathrm{d}x.$$

*A skew-symmetric matrix has purely imaginary eigenvalues and satisfies $\boldsymbol{z}^\top \boldsymbol{K}^{\mathrm{conv}} \boldsymbol{z} = 0$ for every $\boldsymbol{z}$. It transports energy between modes and removes none. All the dissipation in the discrete problem comes from the diffusive block, and when $\nu$ is small there is almost none.*

### What the modified test function does

Replace $\phi_m$ by

$$\psi_m = \phi_m + \tau\, a\, \partial_x \phi_m.$$

The projected convective term becomes

$$\int_0^1 \psi_m\, a\,\partial_x u\,\mathrm{d}x = \underbrace{\int_0^1 \phi_m\, a\,\partial_x u\,\mathrm{d}x}_{\text{as before, skew}} + \underbrace{\tau \int_0^1 a^2\,\partial_x\phi_m\,\partial_x u\,\mathrm{d}x}_{\text{new, and symmetric}}.$$

*Look at the new term: it has the same shape as the diffusive term $\nu\int \phi_m' u'$, with $\nu$ replaced by $\tau a^2$. The upwind test function has added an artificial diffusivity of exactly that size, and it acts only along the direction of transport.*

### Choosing $\tau$

For the one-dimensional problem of slide 51, with cell Péclet number $\mathrm{Pe} = a\Delta x/(2\nu)$, the choice

$$\tau = \frac{\Delta x}{2a}\left(\coth \mathrm{Pe} - \frac{1}{\mathrm{Pe}}\right)$$

makes the scheme reproduce the exact solution at the nodes.

- $\mathrm{Pe} \to 0$: $\tau \to 0$, and nothing is added where nothing is needed
- $\mathrm{Pe} \to \infty$: $\tau \to \Delta x/(2a)$, full upwinding

**What it costs**

| | |
|---|---|
| gained | a stable scheme at any Pe, with the added dissipation aligned with the flow |
| lost | the symmetry of $\boldsymbol{K}$, and with it the energy-norm optimality of Cea's lemma |
| lost | simplicity: $\tau$ must be tuned, and a poor choice smears the solution |

**The same idea for reduced models**

In model reduction the analogous choice is **LSPG**, Least-Squares Petrov–Galerkin. Instead of a modified test function it takes the test space

$$\mathcal{W} = \boldsymbol{J}\,\boldsymbol{\Phi}_{n_r}, \qquad \boldsymbol{J} = \frac{\partial \boldsymbol{r}}{\partial \boldsymbol{u}},$$

the Jacobian of the discrete residual applied to the basis, which is exactly the choice that minimises $\|\boldsymbol{r}\|_2$ over the reduced subspace.

- Galerkin minimises the error in the energy norm, when one exists
- LSPG minimises the residual of the discretised problem, which always exists
- the price is the same: no symmetry, and a result that depends on the discretisation and on the time step

*The pattern is worth stating once for both: whenever the operator is not symmetric, we give up optimality in the error and settle for optimality in the residual. Petrov–Galerkin is the general name for that trade.*

**Slide 40** The weak form and integration by parts

**Step 1. Why we cannot leave the second derivative alone.** Take a hat function $\phi_j$ on a uniform mesh. Its derivative is piecewise constant,

$$\phi_j'(x) = \begin{cases} +1/\Delta x & x \in (x_{j-1}, x_j) \\ -1/\Delta x & x \in (x_j, x_{j+1}) \\ 0 & \text{elsewhere,} \end{cases}$$

so $\phi_j''$ contains Dirac deltas at the three nodes and $\int \phi_m \phi_n''\,\mathrm{d}x$ is meaningless in the classical sense.

**Step 2. Integrate by parts, keeping every term.**

$$-\int_0^1 \psi\,\partial_{xx} u\,\mathrm{d}x = -\Big[\psi\,\partial_x u\Big]_0^1 + \int_0^1 \partial_x \psi\,\partial_x u\,\mathrm{d}x$$

*Write the boundary term first and do not erase it. Everything about boundary conditions lives in that bracket.*

**Step 3. Discuss the bracket, case by case.**

| at that boundary | we prescribe | the bracket |
|---|---|---|
| Dirichlet | $u$ | vanishes, because we *choose* $\psi = 0$ there |
| Neumann | $\partial_x u = g$ | survives and becomes known data, $-\psi(1)g$ |
| Robin | $\partial_x u + \alpha u = g$ | survives and contributes to both sides |

*This is the whole content of the words "essential" and "natural". A Dirichlet condition is imposed by restricting the space. A Neumann condition is not imposed at all: it appears by itself, out of the integration by parts.*

**Step 4. Count the derivatives.** The left-hand side asked for two derivatives of $u$ and none of $\psi$. The right-hand side asks for one of each. Symmetry restored, and piecewise linear functions now qualify.

$$a(\psi, u) = \int_0^1 \partial_x \psi \, \partial_x u \, \mathrm{d}x = a(u, \psi) \quad \Longrightarrow \quad \boldsymbol{K} = \boldsymbol{K}^\top.$$

**Slide 43** Spectral Galerkin: why the operators collapse

Keep $\phi_n = \sqrt{2}\sin(n\pi x)$ on $(0, 1)$, which satisfies $\phi_n(0) = \phi_n(1) = 0$.

**Step 1. Mass matrix.** Already done on slide 16: $A_{mn} = \delta_{mn}$.

**Step 2. Stiffness matrix, the direct way.**

$$\phi_n''(x) = -(n\pi)^2 \sqrt{2}\sin(n\pi x) = -(n\pi)^2 \phi_n(x)$$

so

$$K_{mn} = \nu \langle \phi_m, \phi_n'' \rangle_\Omega = -\nu(n\pi)^2 \langle \phi_m, \phi_n \rangle_\Omega = -\nu(n\pi)^2 \delta_{mn}.$$

For $n_b = 3$,

$$\boldsymbol{K} = -\nu\pi^2 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 9 \end{bmatrix}.$$

**Step 3. Say why, not just what.** $\phi_n$ are the eigenfunctions of $\partial_{xx}$ with homogeneous Dirichlet conditions. Projecting an operator onto its own eigenfunctions is a diagonalisation, so the three diffusion equations decouple:

$$\dot{a}_1 = -\nu\pi^2 a_1, \qquad \dot{a}_2 = -4\nu\pi^2 a_2, \qquad \dot{a}_3 = -9\nu\pi^2 a_3.$$

Each mode decays independently, at a rate proportional to $n^2$.

**Step 4. The same calculation done by parts, as a check.**

$$\begin{aligned} \langle \phi_m, \phi_n'' \rangle_\Omega &= \underbrace{[\phi_m \phi_n']_0^1}_{=0 \text{ since } \phi_m(0)=\phi_m(1)=0} - \int_0^1 \phi_m' \phi_n' \, \mathrm{d}x \\ &= -2mn\pi^2 \int_0^1 \cos(m\pi x)\cos(n\pi x)\, \mathrm{d}x = -(n\pi)^2 \delta_{mn} \end{aligned}$$

*Two routes, same answer. The second one is the one that generalises to finite elements, because it never differentiates twice.*

The nonlinear term does *not* diagonalise, and slide 33 shows why: its selection rules connect $m$, $n$ and $k$ rather than forcing them equal. Modes exchange energy through the triads. That is the whole of nonlinear dynamics in one sentence, and it is worth saying it that way.

**Slide 44a** Why finite elements need the weak form

*Start with the obstacle, not with the solution. Ask the class to draw a piecewise linear function and then to draw its second derivative.*

A hat function is continuous but its slope jumps at every node. Its first derivative is a piecewise constant, and its second derivative is a sum of Dirac deltas sitting on the nodes:

$$\phi_j \in C^0, \qquad \phi_j' \text{ piecewise constant}, \qquad \phi_j'' = \frac{1}{\Delta x}(\delta_{x_{j-1}} - 2\delta_{x_j} + \delta_{x_{j+1}}).$$

So $\langle \phi_m, \nu \phi_n'' \rangle_\Omega$, written as it stands, is a product of distributions and the strong residual $\mathcal{R}$ is not a function we can integrate against anything.

**The repair.** Integrate the second-derivative term by parts once. On $\Omega = (0,1)$, with $\phi_m$ vanishing at both ends,

$$\int_0^1 \phi_m \, \nu \, \phi_n'' \, \mathrm{d}x = \underbrace{\Big[\nu \, \phi_m \phi_n'\Big]_0^1}_{=0 \text{ because } \phi_m(0)=\phi_m(1)=0} - \nu \int_0^1 \phi_m' \, \phi_n' \, \mathrm{d}x \;=\; -\nu \, K_{mn}.$$

Now each factor carries only one derivative, both are piecewise constants, and the integral is an ordinary finite sum over the elements.

**The trade, in one line.** Integration by parts moves one derivative from the trial function to the test function. The price is that the boundary term must be dealt with; the reward is that the space needs only *one* square integrable derivative, which is exactly what a hat function has. Spaces of functions with one square integrable derivative are called $H^1$.

*This is why the finite element method is stated weakly and the spectral method need not be: a global sine is infinitely differentiable, so nothing forces the integration by parts there. It is done anyway, out of habit and because it makes the operator symmetric.*

Two derivatives are not conserved, one is moved. A common confusion is to think the weak form is an approximation. It is not: for a smooth $u$ the weak and strong statements are equivalent. What the weak form does is *enlarge* the set of admissible functions to include ones the strong form cannot even accept.

**Slide 44b** The hat function, written out

On a mesh $0 = x_0 < x_1 < \cdots < x_N = 1$ with uniform spacing $\Delta x$:

$$\phi_j(x) = \begin{cases} \dfrac{x - x_{j-1}}{\Delta x}, & x \in [x_{j-1}, x_j], \\ \dfrac{x_{j+1} - x}{\Delta x}, & x \in [x_j, x_{j+1}], \\ 0 & \text{elsewhere.} \end{cases}$$

Three properties, all worth writing on the board and none of them shared by the sines of the previous slides.

**1. Nodal.** $\phi_j(x_i) = \delta_{ij}$. Therefore

$$v_{n_b}(x) = \sum_n a_n \phi_n(x) \qquad \Longrightarrow \qquad v_{n_b}(x_i) = a_i.$$

*The coefficients* are *the nodal values. In the spectral method the coefficients were amplitudes of global waves and had no pointwise meaning. Here you can read the solution straight off the vector* $\boldsymbol{a}$.

**2. Local support.** $\phi_j$ is nonzero only on the two elements that touch node $j$. Hence

$$\langle \phi_m, \phi_n \rangle_\Omega = 0 \quad \text{whenever} \quad |m - n| > 1,$$

and the Gram matrix is tridiagonal rather than full. This is the single reason the finite element method scales.

**3. Partition of unity.** $\sum_j \phi_j(x) = 1$ for every $x$. So the constant function is represented exactly, and any smooth function is interpolated to $O(\Delta x^2)$.

**The one comparison to draw.** On the left, three sines on $[0,1]$: each one is nonzero everywhere, the Gram matrix is diagonal, and every coefficient sees the whole domain. On the right, three hats: each is nonzero on two elements, the Gram matrix is tridiagonal, and every coefficient sees only its own neighbourhood. Orthogonality has been traded for sparsity.

**Slide 45a** The reference element and the change of variables

Every element is mapped to one master interval, so the integrals are computed once and reused. With $e = [x_j, x_{j+1}]$ and

$$\xi = \frac{x - x_j}{\Delta x} \in [0,1], \qquad x = x_j + \xi\,\Delta x, \qquad \mathrm{d}x = \Delta x\,\mathrm{d}\xi, \qquad \frac{\mathrm{d}}{\mathrm{d}x} = \frac{1}{\Delta x}\frac{\mathrm{d}}{\mathrm{d}\xi},$$

the restrictions of the two nonzero basis functions to the element are

$$\phi_1^e(\xi) = 1 - \xi, \qquad \phi_2^e(\xi) = \xi, \qquad \frac{\mathrm{d}\phi_1^e}{\mathrm{d}x} = -\frac{1}{\Delta x}, \qquad \frac{\mathrm{d}\phi_2^e}{\mathrm{d}x} = +\frac{1}{\Delta x}.$$

*Write the two factors of $\Delta x$ separately and keep track of them. Every mass-type integral picks up one factor of $\Delta x$ from $\mathrm{d}x$. Every derivative picks up one factor of $1/\Delta x$. That bookkeeping alone predicts the $\Delta x$ and $1/\Delta x$ in front of the two element matrices, before a single integral is evaluated.*

| term | factors of $\Delta x$ | result |
|---|---|---|
| $\int \phi_a^e \phi_b^e\,\mathrm{d}x$ | $\Delta x$ from $\mathrm{d}x$ | $\propto \Delta x$ |
| $\int (\phi_a^e)'(\phi_b^e)'\,\mathrm{d}x$ | $\frac{1}{\Delta x}\cdot\frac{1}{\Delta x}\cdot\Delta x$ | $\propto \Delta x^{-1}$ |
| $\int \phi_a^e (\phi_b^e)'\,\mathrm{d}x$ | $\frac{1}{\Delta x}\cdot\Delta x$ | $\propto 1$ |

The local index $a \in \{1,2\}$ and the global index $j$ are different things. The map between them is the *connectivity*: element $e$ owns global nodes $(e-1, e)$ on a one-dimensional mesh. Every assembly bug is a connectivity bug.

**Slide 45** The finite element matrices, assembled by hand

Work on one element $e = [x_j, x_{j+1}]$ of length $\Delta x$, with local coordinate $\xi = (x - x_j)/\Delta x \in [0,1]$ and the two nonzero basis functions restricted to it

$$\phi_1^e(\xi) = 1 - \xi, \qquad \phi_2^e(\xi) = \xi, \qquad \frac{\mathrm{d}\phi_1^e}{\mathrm{d}x} = -\frac{1}{\Delta x}, \qquad \frac{\mathrm{d}\phi_2^e}{\mathrm{d}x} = +\frac{1}{\Delta x}.$$

**Step 1. Element mass matrix, all four entries.**

$$A_{11}^e = \int_0^1 (1-\xi)^2\,\Delta x\,\mathrm{d}\xi = \tfrac{\Delta x}{3}, \qquad A_{12}^e = \int_0^1 (1-\xi)\xi\,\Delta x\,\mathrm{d}\xi = \tfrac{\Delta x}{6},$$
$$A_{21}^e = A_{12}^e = \tfrac{\Delta x}{6}, \qquad A_{22}^e = \int_0^1 \xi^2\,\Delta x\,\mathrm{d}\xi = \tfrac{\Delta x}{3}.$$

$$\Rightarrow \qquad \boldsymbol{A}^e = \frac{\Delta x}{6}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

**Step 2. Element stiffness matrix.** The derivatives are constants, so the integrals are trivial:

$$K_{11}^e = \int_{x_j}^{x_{j+1}} \left(\frac{-1}{\Delta x}\right)^2 \mathrm{d}x = \frac{1}{\Delta x}, \qquad K_{12}^e = -\frac{1}{\Delta x}, \qquad \Rightarrow \qquad \boldsymbol{K}^e = \frac{1}{\Delta x}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

*Note $\boldsymbol{K}^e$ is singular: a constant function has zero derivative. It is the boundary conditions that make the assembled system solvable.*

**Step 3. Assemble, on a three-element mesh.** Take $\Omega = (0,1)$ with $\Delta x = 1/3$ and nodes $x_0, \dots, x_3$. Element 1 contributes to nodes $(0,1)$, element 2 to $(1,2)$, element 3 to $(2,3)$, each adding its $2 \times 2$ block on the diagonal:

$$\boldsymbol{A} = \frac{\Delta x}{6}\begin{bmatrix} 2 & 1 & 0 & 0 \\ 1 & 2+2 & 1 & 0 \\ 0 & 1 & 2+2 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix} = \frac{\Delta x}{6}\begin{bmatrix} 2 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}, \qquad \boldsymbol{K} = \frac{1}{\Delta x}\begin{bmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{bmatrix}$$

*The interior rows of $\boldsymbol{K}$ read $(-1, 2, -1)/\Delta x$. That is the finite difference Laplacian multiplied by $\Delta x$, arrived at from a completely different starting point. Worth pausing on.*

**Step 4. Apply the Dirichlet conditions.** Delete the rows and columns of the constrained nodes, or lift as in the tutorial. With both ends constrained, only the $2 \times 2$ interior block survives:

$$\boldsymbol{A}_{\text{int}} = \frac{\Delta x}{6}\begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}, \qquad \boldsymbol{K}_{\text{int}} = \frac{1}{\Delta x}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}.$$

$\boldsymbol{A}$ is not diagonal, so the semi-discrete system is $\boldsymbol{A}\dot{\boldsymbol{a}} = \dots$ and every time step needs a solve. Mass lumping replaces $\boldsymbol{A}$ by the diagonal matrix holding its row sums,

$$\boldsymbol{A}_{\text{lumped}} = \frac{\Delta x}{6}\,\text{diag}(3, 6, 6, 3),$$

which makes the scheme explicit at the cost of accuracy. Ask the class what has been assumed: that the mass of each element can be split between its two nodes and treated as concentrated there.

**Slide 45b** Assembly, with the indices written out

*This is the step students skip and then cannot debug. Do it slowly, on a three-element mesh, and write the loop as it appears in the code.*

Mesh: $\Omega = (0,1)$, $\Delta x = 1/3$, nodes $x_0, x_1, x_2, x_3$, elements $e = 1, 2, 3$ owning the node pairs $(0,1)$, $(1,2)$, $(2,3)$.

**The assembly loop, in words and in symbols.**

$$\text{for each element } e: \qquad A_{g(e,a)\,g(e,b)} \;\mathrel{+}=\; A^e_{ab}, \qquad a, b \in \{1, 2\},$$

where $g(e,a)$ is the global node number of local node $a$ of element $e$; here $g(e,1) = e - 1$ and $g(e,2) = e$.

Written out for the mass matrix, one element at a time:

$$\underbrace{\frac{\Delta x}{6}\begin{bmatrix} 2 & 1 & & \\ 1 & 2 & & \\ & & & \\ & & & \end{bmatrix}}_{e=1} + \underbrace{\frac{\Delta x}{6}\begin{bmatrix} & & & \\ & 2 & 1 & \\ & 1 & 2 & \\ & & & \end{bmatrix}}_{e=2} + \underbrace{\frac{\Delta x}{6}\begin{bmatrix} & & & \\ & & & \\ & & 2 & 1 \\ & & 1 & 2 \end{bmatrix}}_{e=3} = \frac{\Delta x}{6}\begin{bmatrix} 2 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}$$

*Point at the two interior diagonal entries. The 4 is $2+2$: node 1 is shared by elements 1 and 2, so it receives a contribution from each. That is the whole of assembly.*

**Why the result is banded.** Row $m$ of $\boldsymbol{A}$ is nonzero only in the columns $n$ for which $\phi_m$ and $\phi_n$ share an element. On this mesh that is $n \in \{m-1, m, m+1\}$. Nothing about the physics produced the band; it came from the supports.

**Slide 45c** The convective and the quadratic terms, element by element

The mass and stiffness matrices are the easy ones because they are symmetric. The two terms that come from the convection are not, and they are where the signs go wrong.

**The convective block.** With $h(x)$ the lifting evaluated on the element and $B_{mn} = \langle \phi_m, h\,\phi_n' \rangle_\Omega$, on one element

$$B^e_{ab} = \int_0^1 \phi^e_a(\xi)\, h(\xi)\, \frac{\mathrm{d}\phi^e_b}{\mathrm{d}x}\, \Delta x\, \mathrm{d}\xi = \frac{\mathrm{d}\phi^e_b}{\mathrm{d}x}\, \Delta x \int_0^1 \phi^e_a\, h\, \mathrm{d}\xi.$$

Taking $h$ constant on the element and equal to its midpoint value $h_e$, and using $\int_0^1 \phi^e_1\, \mathrm{d}\xi = \int_0^1 \phi^e_2\, \mathrm{d}\xi = \frac{1}{2}$:

$$\boldsymbol{B}^e = \frac{h_e}{2} \begin{bmatrix} -1 & +1 \\ -1 & +1 \end{bmatrix}.$$

*Note that $\boldsymbol{B}^e$ is* not *symmetric, and that its rows are equal. The asymmetry is not a mistake: convection has a direction, and a symmetric operator cannot represent one. This is the same asymmetry that ruins Cholesky in the tutorial solver, and the same one that will make Galerkin oscillate at high Péclet number.*

**The quadratic block.** With $C_{mnk} = -\langle \phi_m, \phi_n \phi_k' \rangle_\Omega$, on one element the local object is a $2 \times 2 \times 2$ tensor,

$$C^e_{abc} = -\frac{\mathrm{d}\phi^e_c}{\mathrm{d}x}\, \Delta x \int_0^1 \phi^e_a \phi^e_b\, \mathrm{d}\xi = \mp \frac{1}{\Delta x}\, \Delta x \int_0^1 \phi^e_a \phi^e_b\, \mathrm{d}\xi,$$

with the sign taken from $\mathrm{d}\phi^e_c/\,\mathrm{d}x = \mp 1/\Delta x$. Using $\int_0^1 (\phi^e_1)^2 = \int_0^1 (\phi^e_2)^2 = \frac{1}{3}$ and $\int_0^1 \phi^e_1 \phi^e_2 = \frac{1}{6}$:

$$C^e_{ab1} = +\frac{1}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \qquad C^e_{ab2} = -\frac{1}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

*Two things to observe. First, $\Delta x$ has cancelled: the quadratic term is mesh-size independent, which is why it does not stiffen the system the way diffusion does. Second, the two slices differ only by a sign, because the two basis functions have opposite slopes.*

The tensor is assembled with the *same* three loops as the matrices, but now over three local indices, and it is a $2 \times 2 \times 2$ block that is scattered into an $n_b \times n_b \times n_b$ array. In the tutorial this is done in one `einsum` over quadrature points, which is the same arithmetic written differently. If the reduced model blows up, this is the first object to check, and the fastest check is $C_{mnk} = C_{nmk}$, which holds because $m$ and $n$ enter only through the product $\phi_m \phi_n$. A violation means two indices have been transposed somewhere in the assembly.

**Slide 50** The lifting, done term by term

The problem is

$$\partial_t u + u\, \partial_x u = \nu\, \partial_{xx} u + \alpha f(x), \qquad u(0,t) = 1, \quad u(1,t) = 0.$$

**Step 1. Choose $h$ and check it.** Take $h(x) = 1 - x$. Then $h(0) = 1$, $h(1) = 0$, $h' = -1$, $h'' = 0$. Set $u = h + v$, so $v(0,t) = v(1,t) = 0$ as required, and since $u(x,0) = 1 - x = h(x)$, also $v(x,0) = 0$.

**Step 2. Substitute into each term separately.** Write the four terms in a column, and next to each its value after substitution:

| | |
|---|---|
| $\partial_t u = \partial_t v$ | $h$ does not depend on $t$ |
| $u\, \partial_x u = (v + 1 - x)(\partial_x v - 1)$ | because $h' = -1$ |
| $\nu\, \partial_{xx} u = \nu\, \partial_{xx} v$ | because $h'' = 0$ |
| $\alpha f(x) = \alpha f(x)$ | unchanged |

**Step 3. Expand the convective term completely.**

$$(v+1-x)(\partial_x v - 1) = \underbrace{v\,\partial_x v}_{\text{quadratic}} \underbrace{-\,v}_{\text{linear}} \underbrace{+(1-x)\partial_x v}_{\text{linear}} \underbrace{-(1-x)}_{\text{constant}}$$

**Step 4. Move everything to the right-hand side and sort by degree.**

$$\partial_t v = \underbrace{(1-x)+\alpha f(x)}_{\text{constant in } v} + \underbrace{\nu\,\partial_{xx} v + v - (1-x)\partial_x v}_{\text{linear in } v} \underbrace{-\,v\,\partial_x v}_{\text{quadratic in } v}$$

**Three observations to put in a box on the board.**

1. The structure is exactly the template of slide 32, before any basis has been chosen. The basis will only turn functions into matrices.
2. The dependence on $\nu$ and on $\alpha$ is *affine*, and it is affine by construction rather than by luck. This is what will let us precompute everything offline.
3. The lifting produced two extra linear terms, $+v$ and $-(1-x)\partial_x v$. Students forget these constantly. They come from differentiating $h$, not from the original equation.

**Slide 52** The projection of the tutorial problem

Project the lifted equation on $\phi_n = \sqrt{2}\sin(n\pi x)$, one term at a time. Write the target first:

$$\dot{\boldsymbol{a}} = \boldsymbol{c}_0 + \boldsymbol{L}\boldsymbol{a} + \boldsymbol{C} : (\boldsymbol{a} \otimes \boldsymbol{a}).$$

**Time derivative.** $\langle \phi_m, \phi_n \rangle_\Omega = \delta_{mn}$, so the left-hand side is simply $\dot{a}_m$. No mass matrix to invert.

**Constant terms.** Two contributions, both computable once:

$$c_{0,m} = \underbrace{\langle \phi_m, 1-x \rangle_\Omega}_{\text{from the lifting}} + \alpha \underbrace{\langle \phi_m, f \rangle_\Omega}_{\text{from the source}}.$$

The first has a closed form worth deriving on the board:

$$\int_0^1 (1-x)\sqrt{2}\sin(m\pi x)\,\mathrm{d}x = \frac{\sqrt{2}}{m\pi}.$$

*It decays like $1/m$, slowly, because the odd extension of $1-x$ has a jump at the origin. This is the same phenomenon as slide 16 and it is why the lifting term needs a few more modes than one might expect.*

**Linear terms.** Three pieces:

$$L_{mn} = \underbrace{-\nu(n\pi)^2\delta_{mn}}_{\text{diffusion}} + \underbrace{\delta_{mn}}_{\text{from } +v} \underbrace{-\langle \phi_m, (1-x)\phi_n' \rangle_\Omega}_{\text{from } -(1-x)\partial_x v}$$

Only the last one is a full matrix. Notice it is *not* symmetric, which is correct: a convective operator should not be.

**Quadratic term.** Exactly the tensor of slide 33, with the closed form derived there:

$$C_{mnk} = -\frac{\sqrt{2}\,k\pi}{2}\left(\delta_{|m-n|,k} - \delta_{m+n,k}\right).$$

**Initial condition.** $v(x,0) = 0$ gives $\boldsymbol{a}(0) = \boldsymbol{0}$, so the reduced model starts from the origin of its own state space. Worth remarking that this is a gift of the lifting: with a different $h$ we would have had to project the initial condition.

**Cost bookkeeping, on the board next to the result.**

| object | size | computed |
|---|---|---|
| $\boldsymbol{c}_0$ | $n_b$ | once |
| $\boldsymbol{L}$ | $n_b^2$ | once, split into $\nu \boldsymbol{L}^{\text{diff}} + \boldsymbol{L}^{\text{conv}}$ |
| $\boldsymbol{C}$ | $n_b^3$ | once |
| one right-hand side | $\mathcal{O}(n_b^2) + \mathcal{O}(n_b^3)$ | every time step |

For $n_b = 16$: 256 entries in $\boldsymbol{L}$ and 4096 in $\boldsymbol{C}$. For $n_b = 64$ the tensor already holds a quarter of a million numbers, which is the practical reason spectral Galerkin is not pushed to large $n_b$ in nonlinear problems.

## 3 Section 4: POD and POD-Galerkin

**Slide 58** From the optimisation problem to the eigenproblem

*The question of this slide: yes, this is a Lagrange multiplier calculation, and it is worth doing twice. First in finite dimension, where everything is familiar, then in function space, where it looks identical.*

**Part A. The discrete version, which students already know**

**Step 0. Where the quadratic form comes from, because it is not the starting point.**
*Do not open with* $\max \boldsymbol{\phi}^\top \boldsymbol{C}\boldsymbol{\phi}$. *Nobody would write that down unprompted. Open with the question anyone would ask: what single direction reconstructs my data best? The quadratic form is what that question turns into, and the two lines in between are the ones that make the word "energy" legitimate.*

The honest cost function is a reconstruction error. Look for one unit vector $\boldsymbol{\phi}$ and, for each snapshot, one coefficient $a_k$, so that $a_k\boldsymbol{\phi}$ is as close to $\boldsymbol{d}_k$ as possible:

$$\min_{\boldsymbol{\phi},\ \{a_k\}} \quad J = \sum_{k=1}^{n_t} \|\boldsymbol{d}_k - a_k\boldsymbol{\phi}\|^2 \qquad \text{subject to} \qquad \boldsymbol{\phi}^\top\boldsymbol{\phi} = 1.$$

**First eliminate the coefficients.** For a fixed $\boldsymbol{\phi}$, differentiate one term with respect to $a_k$:

$$\frac{\partial}{\partial a_k}\|\boldsymbol{d}_k - a_k\boldsymbol{\phi}\|^2 = -2\,\boldsymbol{\phi}^\top\boldsymbol{d}_k + 2a_k\underbrace{\boldsymbol{\phi}^\top\boldsymbol{\phi}}_{=1} = 0 \qquad \Longrightarrow \qquad \boxed{a_k = \boldsymbol{\phi}^\top\boldsymbol{d}_k}$$

*Worth pausing on. The projection coefficient was not assumed, it was derived. Least squares hands it to you the moment you ask for the closest multiple of* $\boldsymbol{\phi}$*, which is the same statement as the normal equations of Section 2 with a single basis function.*

**Then substitute it back, and Pythagoras appears.**

$$\begin{aligned}\|\boldsymbol{d}_k - (\boldsymbol{\phi}^\top\boldsymbol{d}_k)\boldsymbol{\phi}\|^2 &= \boldsymbol{d}_k^\top\boldsymbol{d}_k - 2(\boldsymbol{\phi}^\top\boldsymbol{d}_k)^2 + (\boldsymbol{\phi}^\top\boldsymbol{d}_k)^2\underbrace{\boldsymbol{\phi}^\top\boldsymbol{\phi}}_{=1}\\ &= \underbrace{\|\boldsymbol{d}_k\|^2}_{\text{total}} - \underbrace{(\boldsymbol{\phi}^\top\boldsymbol{d}_k)^2}_{\text{captured}}.\end{aligned}$$

$$\boxed{\underbrace{\|\boldsymbol{d}_k\|^2}_{\text{energy of the snapshot}} = \underbrace{(\boldsymbol{\phi}^\top\boldsymbol{d}_k)^2}_{\text{energy captured by }\boldsymbol{\phi}} + \underbrace{\|\boldsymbol{d}_k - (\boldsymbol{\phi}^\top\boldsymbol{d}_k)\boldsymbol{\phi}\|^2}_{\text{energy left over}}}$$

**Now sum over the snapshots.**

$$J(\boldsymbol{\phi}) = \sum_k \|\boldsymbol{d}_k\|^2 - \sum_k(\boldsymbol{\phi}^\top\boldsymbol{d}_k)^2 = \underbrace{\sum_k \|\boldsymbol{d}_k\|^2}_{\text{fixed by the data}} - n_t\ \boldsymbol{\phi}^\top\boldsymbol{C}\boldsymbol{\phi}, \qquad \boldsymbol{C} = \frac{1}{n_t}\boldsymbol{D}\boldsymbol{D}^\top,$$

because $\sum_k(\boldsymbol{\phi}^\top\boldsymbol{d}_k)^2 = \boldsymbol{\phi}^\top\boldsymbol{D}\boldsymbol{D}^\top\boldsymbol{\phi}$.

The first term does not contain $\boldsymbol{\phi}$ at all: it is the total energy of the dataset, and no choice of basis can change it. So

$$\boxed{\min_{\boldsymbol{\phi}}\ (\text{energy left over}) \iff \max_{\boldsymbol{\phi}}\ (\text{energy captured}) \iff \max_{\boldsymbol{\phi}}\ \boldsymbol{\phi}^\top\boldsymbol{C}\boldsymbol{\phi}}$$

**This is why "energy" is the right word and not a metaphor.** $\|\boldsymbol{d}_k\|^2$ is the discrete $L^2$ energy of one snapshot, the identity above splits it exactly into a captured part and a leftover part, and nothing was approximated in getting there. Every later statement inherits its meaning from this one line: $\sigma_r^2$ is the energy captured by mode $r$, $E_{n_r} = \sum_{r\le n_r}\sigma_r^2 / \sum_r \sigma_r^2$ is the fraction captured by the first $n_r$, and $1 - E_{n_r}$ is the fraction left over. Those are identities, not rules of thumb.

The same calculation with $n_r$ directions instead of one gives $J = \sum_k \|\boldsymbol{d}_k\|^2 - \sum_{r\le n_r}\sigma_r^2 = \sum_{r>n_r}\sigma_r^2$, which is Eckart–Young. So the optimality result later in the section is not a separate theorem to be believed: it is this same step, done with a projector rather than a single direction. The only ingredient that has to be added is that the optimal $n_r$-dimensional space is spanned by the leading eigenvectors, which is what the sequence of maximisations in Step 5 establishes.

**Step 1. State the constrained problem.**

$$\max_{\boldsymbol{\phi}} \ \boldsymbol{\phi}^\top \boldsymbol{C}\boldsymbol{\phi} \qquad \text{subject to} \qquad \boldsymbol{\phi}^\top\boldsymbol{\phi} = 1.$$

*Ask first why the constraint is needed. Without it, doubling $\boldsymbol{\phi}$ quadruples the objective and the problem has no maximum. The constraint fixes the scale so that only the* direction *is being chosen. Note that the constraint was already there in Step 0, where it was needed to make $a_k = \boldsymbol{\phi}^\top\boldsymbol{d}_k$ come out clean.*

**Step 2. Build the Lagrangian.**

$$\mathcal{J}(\boldsymbol{\phi}, \lambda) = \boldsymbol{\phi}^\top\boldsymbol{C}\boldsymbol{\phi} - \lambda(\boldsymbol{\phi}^\top\boldsymbol{\phi} - 1)$$

**Step 3. Differentiate.** Using $\partial(\boldsymbol{\phi}^\top\boldsymbol{C}\boldsymbol{\phi})/\partial\boldsymbol{\phi} = 2\boldsymbol{C}\boldsymbol{\phi}$ for symmetric $\boldsymbol{C}$, and $\partial(\boldsymbol{\phi}^\top\boldsymbol{\phi})/\partial\boldsymbol{\phi} = 2\boldsymbol{\phi}$,

$$\frac{\partial\mathcal{J}}{\partial\boldsymbol{\phi}} = 2\boldsymbol{C}\boldsymbol{\phi} - 2\lambda\boldsymbol{\phi} = \boldsymbol{0} \qquad \Longrightarrow \qquad \boxed{\boldsymbol{C}\boldsymbol{\phi} = \lambda\boldsymbol{\phi}}$$

*The stationarity condition of a constrained quadratic maximisation* is *an eigenvalue problem. Nothing else needed to be assumed.*

**Step 4. Identify the multiplier.** Multiply the eigenvalue equation by $\boldsymbol{\phi}^\top$ and use the constraint:

$$\boldsymbol{\phi}^\top\boldsymbol{C}\boldsymbol{\phi} = \lambda\,\boldsymbol{\phi}^\top\boldsymbol{\phi} = \lambda.$$

So $\lambda$ is not an auxiliary unknown to be discarded: it *is* the value of the objective. Maximising means picking the largest eigenvalue, and the eigenvalues rank the modes.

**Step 5. The second mode, and why orthogonality is automatic.** Repeat the problem with one extra constraint, $\boldsymbol{\phi}^\top\boldsymbol{\phi}_1 = 0$:

$$\mathcal{J}_2 = \boldsymbol{\phi}^\top\boldsymbol{C}\boldsymbol{\phi} - \lambda(\boldsymbol{\phi}^\top\boldsymbol{\phi} - 1) - \mu\,\boldsymbol{\phi}^\top\boldsymbol{\phi}_1.$$

Stationarity gives $2\boldsymbol{C}\boldsymbol{\phi} - 2\lambda\boldsymbol{\phi} - \mu\boldsymbol{\phi}_1 = \boldsymbol{0}$. Multiply on the left by $\boldsymbol{\phi}_1^\top$ and use $\boldsymbol{C}\boldsymbol{\phi}_1 = \lambda_1\boldsymbol{\phi}_1$ together with $\boldsymbol{\phi}_1^\top\boldsymbol{\phi} = 0$:

$$2\lambda_1\underbrace{\boldsymbol{\phi}_1^\top\boldsymbol{\phi}}_{=0} - 2\lambda\underbrace{\boldsymbol{\phi}_1^\top\boldsymbol{\phi}}_{=0} - \mu = 0 \qquad \Longrightarrow \qquad \mu = 0,$$

so the second problem reduces to the same eigenproblem and $\boldsymbol{\phi}_2$ is the eigenvector of the second largest eigenvalue. *Orthogonality of the POD modes was not imposed as a matter of taste. It falls out of the sequence of maximisations, because $\boldsymbol{C}$ is symmetric.*

**Part B. The same calculation in function space**

Now $u(x,\xi)$ is a random field and we look for a function $\phi(x)$.

**Step 0, again.** The cost function is the same reconstruction error, with an expectation in place of the sum over snapshots:

$$\min_{\phi} \; E\Big[\; \|u - a(\xi)\,\phi\|_{L^2}^2 \;\Big] \qquad \text{subject to} \qquad \int_\Omega \phi^2 \,\mathrm{d}\Omega = 1.$$

Minimising over $a(\xi)$ first gives $a(\xi) = \langle u, \phi\rangle_\Omega$, exactly as before, and substituting it back gives the same split,

$$E[\,\|u\|_{L^2}^2\,] = E[|\,\langle u,\phi\rangle_\Omega\,|^2] + E[\,\|u - a\phi\|_{L^2}^2\,],$$

whose first term is fixed by the data. So maximising $E[|\,\langle u,\phi\rangle_\Omega\,|^2]$ is again the same problem as minimising what is left over, and only that is written from here on.

**Step 1. The objective.**

$$\mathcal{E}[\phi] = E\Big[|\,\langle u,\phi\rangle_\Omega\,|^2\Big] = E\left[\int_\Omega u(x,\xi)\phi(x)\,\mathrm{d}x \int_\Omega u(x',\xi)\phi(x')\,\mathrm{d}x'\right]$$

Exchange expectation and integrals, and define the covariance kernel $C(x,x') = E\{u(x,\xi)u(x',\xi)\}$:

$$\mathcal{E}[\phi] = \int_\Omega\int_\Omega \phi(x)\,C(x,x')\,\phi(x')\,\mathrm{d}x\,\mathrm{d}x'.$$

*Compare with $\boldsymbol{\phi}^\top \boldsymbol{C}\boldsymbol{\phi}$: the same object, a quadratic form built on a symmetric positive kernel.*

**Step 2. The constrained problem and its Lagrangian.**

$$\max_{\phi} \; \mathcal{E}[\phi] \quad \text{subject to} \quad \int_\Omega \phi^2\,\mathrm{d}x = 1, \qquad \mathcal{J}[\phi] = \mathcal{E}[\phi] - \lambda\left(\int_\Omega \phi^2\,\mathrm{d}x - 1\right)$$

**Step 3. Take the variation.** Perturb $\phi \to \phi + \epsilon\eta$ with $\eta$ arbitrary, and collect the terms linear in $\epsilon$. Using the symmetry $C(x,x') = C(x',x)$, the two cross terms are equal, so

$$\frac{\mathrm{d}}{\mathrm{d}\epsilon}\mathcal{J}[\phi+\epsilon\eta]\bigg|_{\epsilon=0} = 2\int_\Omega \eta(x)\left[\int_\Omega C(x,x')\phi(x')\,\mathrm{d}x' - \lambda\phi(x)\right]\mathrm{d}x = 0.$$

**Step 4. Invoke arbitrariness of $\eta$.** The bracket must vanish pointwise, which is the Fredholm eigenvalue problem

$$\boxed{\int_\Omega C(x,x')\,\phi_i(x')\,\mathrm{d}x' = \lambda_i\,\phi_i(x)}$$

*This is the fundamental lemma of the calculus of variations, and it is the only step that differs from Part A. Everything else is identical.*

**Step 5. The same three consequences as in the discrete case.**

- $C$ symmetric and positive semi-definite gives real $\lambda_1 \geq \lambda_2 \geq \cdots \geq 0$ and orthogonal $\phi_i$;
- $\lambda_i = \mathcal{E}[\phi_i]$, so the eigenvalues are the energies;
- $E[a_i a_j] = \lambda_i \delta_{ij}$: the coefficients of the expansion are uncorrelated, which is what "proper" means in the name.

**Part C. Closing the loop with the SVD**

Write the discrete problem in terms of $\boldsymbol{D}$ and note that

$$\boldsymbol{C}\boldsymbol{\phi} = \lambda\boldsymbol{\phi} \quad \text{with} \quad \boldsymbol{C} = \tfrac{1}{n_t}\boldsymbol{D}\boldsymbol{D}^\top \qquad \Longleftrightarrow \qquad \boldsymbol{D}\boldsymbol{D}^\top\boldsymbol{\phi}_r = \sigma_r^2\,\boldsymbol{\phi}_r, \quad \sigma_r^2 = n_t\lambda_r,$$

which are exactly the left singular vectors of $\boldsymbol{D}$. Similarly $\boldsymbol{D}^\top \boldsymbol{D}\,\boldsymbol{\psi}_r = \sigma_r^2 \boldsymbol{\psi}_r$ gives the right singular vectors, and

$$\boldsymbol{\phi}_r = \frac{1}{\sigma_r} \boldsymbol{D} \boldsymbol{\psi}_r.$$

*So "compute the POD" and "compute the SVD of the snapshot matrix" are the same instruction, and the method of snapshots is nothing but the choice of solving the smaller of the two eigenproblems.*

A weighted inner product changes the eigenproblem, not the logic. Repeating Part A with the constraint $\boldsymbol{\phi}^\top \boldsymbol{W}_s \boldsymbol{\phi} = 1$ gives $\boldsymbol{C}\boldsymbol{W}_s\boldsymbol{\phi} = \lambda\boldsymbol{\phi}$, which is a *generalised* eigenvalue problem. It is symmetric in the $\boldsymbol{W}_s$ inner product but not in the Euclidean one, which is why the practical recipe is to factor $\boldsymbol{W}_s = \boldsymbol{L}\boldsymbol{L}^\top$, take the ordinary SVD of $\boldsymbol{L}^\top\boldsymbol{D}$, and map back. On a uniform grid $\boldsymbol{W}_s \propto \boldsymbol{I}$ and the distinction disappears, which is exactly the situation in the tutorial.

**Slide 59** From the kernel to a matrix: what the quadrature does

*Students accept the Fredholm problem and then cannot say where $\boldsymbol{D}\boldsymbol{D}^\top$ came from. Do the substitution explicitly, on three grid points, and the mystery goes away.*

Two approximations, and only two.

**1. The ensemble average becomes a finite sum.**

$$C(x,x') = E\{u(x,\xi)u(x',\xi)\} \;\longrightarrow\; \widehat{C}(x,x') = \frac{1}{n_t}\sum_{k=1}^{n_t} u(x,t_k)\,u(x',t_k).$$

Sampled on the grid, $\widehat{C}(x_i,x_j) = \frac{1}{n_t}\sum_k D_{ik}D_{jk}$, that is

$$\boxed{\boldsymbol{C} = \tfrac{1}{n_t}\boldsymbol{D}\boldsymbol{D}^\top \in \mathbb{R}^{n_s\times n_s}}$$

**2. The integral becomes a quadrature.** With weights $w_i$ collected in $\boldsymbol{W}_s = \mathrm{diag}(w_1,\dots,w_{n_s})$,

$$\int_\Omega C(x_i,x')\phi(x')\,\mathrm{d}x' \;\longrightarrow\; \sum_{j=1}^{n_s} C_{ij}\,w_j\,\phi_j \;=\; (\boldsymbol{C}\,\boldsymbol{W}_s\,\boldsymbol{\phi})_i.$$

**Write it out on three points.** With $n_s = 3$ and weights $(w_1,w_2,w_3)$:

$$\boldsymbol{C}\boldsymbol{W}_s\boldsymbol{\phi} = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix} \begin{bmatrix} w_1 & & \\ & w_2 & \\ & & w_3 \end{bmatrix} \begin{bmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \end{bmatrix} = \begin{bmatrix} C_{11}w_1\phi_1 + C_{12}w_2\phi_2 + C_{13}w_3\phi_3 \\ C_{21}w_1\phi_1 + C_{22}w_2\phi_2 + C_{23}w_3\phi_3 \\ C_{31}w_1\phi_1 + C_{32}w_2\phi_2 + C_{33}w_3\phi_3 \end{bmatrix}$$

Row $i$ is the trapezoidal rule applied to $\int C(x_i,x')\phi(x')\,\mathrm{d}x'$. So

$$\boxed{\boldsymbol{C}\,\boldsymbol{W}_s\,\boldsymbol{\phi}_r = \lambda_r\,\boldsymbol{\phi}_r}$$

*is* the Fredholm problem, evaluated with a quadrature rule. Nothing was assumed beyond the choice of $w_i$.

$\boldsymbol{C}\boldsymbol{W}_s$ is not symmetric in the Euclidean sense, so `numpy.linalg.eigh` on it is wrong. It *is* self-adjoint in the $\boldsymbol{W}_s$ inner product, and the clean way to exploit that is the substitution $\tilde{\boldsymbol{\phi}} = \boldsymbol{W}_s^{1/2}\boldsymbol{\phi}$:

$$\boldsymbol{C}\boldsymbol{W}_s\boldsymbol{\phi} = \lambda\boldsymbol{\phi} \quad\Longleftrightarrow\quad (\boldsymbol{W}_s^{1/2}\boldsymbol{C}\boldsymbol{W}_s^{1/2})\tilde{\boldsymbol{\phi}} = \lambda\tilde{\boldsymbol{\phi}},$$

and the matrix in brackets *is* symmetric. In the tutorial this is done in one line: take the SVD of $\boldsymbol{W}_s^{1/2}\boldsymbol{D}$ and divide the left singular vectors by $\sqrt{w_i}$ afterwards.

**Slide 60** The method of snapshots, derived rather than quoted

*The claim is that two eigenproblems of very different sizes have the same eigenvalues. Prove it in three lines, it is worth the time.*

Take the temporal problem and multiply it by $\boldsymbol{D}$ on the left. Write $\boldsymbol{K} = \boldsymbol{D}^\top \boldsymbol{D} \in \mathbb{R}^{n_t \times n_t}$:

$$\boldsymbol{K}\boldsymbol{\psi} = \sigma^2 \boldsymbol{\psi} \implies \boldsymbol{D}\boldsymbol{D}^\top \boldsymbol{D}\boldsymbol{\psi} = \sigma^2 \boldsymbol{D}\boldsymbol{\psi} \implies \boldsymbol{C}' \underbrace{(\boldsymbol{D}\boldsymbol{\psi})}_{\propto\ \boldsymbol{\phi}} = \sigma^2 (\boldsymbol{D}\boldsymbol{\psi}), \qquad \boldsymbol{C}' = \boldsymbol{D}\boldsymbol{D}^\top.$$

So $\boldsymbol{D}\boldsymbol{\psi}$ is an eigenvector of the *spatial* problem with the same eigenvalue. Normalising,

$$\boxed{\boldsymbol{\phi}_r = \frac{1}{\sigma_r} \boldsymbol{D}\, \boldsymbol{\psi}_r}$$

*Check the normalisation with them: $\|\boldsymbol{D}\boldsymbol{\psi}\|^2 = \boldsymbol{\psi}^\top \boldsymbol{D}^\top \boldsymbol{D}\boldsymbol{\psi} = \sigma^2 \boldsymbol{\psi}^\top \boldsymbol{\psi} = \sigma^2$, so dividing by $\sigma$ gives a unit vector. The factor was not guessed.*

**Why this matters.** $\boldsymbol{C}$ is $n_s \times n_s$ and $\boldsymbol{K}$ is $n_t \times n_t$. A three-dimensional simulation has $n_s \sim 10^7$ and $n_t \sim 10^3$: forming $\boldsymbol{C}$ is impossible and forming $\boldsymbol{K}$ costs $n_t^2 n_s$ multiplications and stores $10^6$ numbers. Sirovich's observation (1987) is exactly this asymmetry.

| | spatial problem | temporal problem |
|---|---|---|
| matrix | $\boldsymbol{C} = \boldsymbol{D}\boldsymbol{D}^\top$ | $\boldsymbol{K} = \boldsymbol{D}^\top \boldsymbol{D}$ |
| size | $n_s \times n_s$ | $n_t \times n_t$ |
| gives | the modes $\boldsymbol{\phi}_r$ directly | the coefficients $\boldsymbol{\psi}_r$ |
| use when | $n_s < n_t$ | $n_s > n_t$, the usual case |

**Slide 62** Optimality: what Eckart–Young actually says

*Do not prove it. State it precisely, and spend the time instead on what it does and does not promise.*

With $P_{n_r}$ the orthogonal projector onto the span of the first $n_r$ modes,

$$\sum_{k=1}^{n_t} \left\| \boldsymbol{d}_k - P_{n_r} \boldsymbol{d}_k \right\|^2 = \sum_{r > n_r} \sigma_r^2,$$

and no other $n_r$-dimensional subspace gives a smaller value.

**Three readings, all worth writing down.**

1. *The error is known in advance.* The tail of the spectrum is computed before any reduced model exists. If $\sum_{r>n_r} \sigma_r^2$ is already too large, no projection method on that space can succeed.
2. *Optimal on this data, in this norm.* Change the snapshots and the optimal space changes. Change the inner product and it changes again. There is no basis that is optimal for a problem, only for a dataset.
3. *Representation is not dynamics.* The statement is about approximating the given columns. It says nothing about the trajectory the reduced model will produce when it is integrated forward, and the two can differ by orders of magnitude.

The most common misuse: choosing $n_r$ from 99.9% of the energy and then being surprised that the reduced model is wrong. The energy criterion bounds the *projection* error, which is the dashed curve in the tutorial. The model error is the solid one. The tutorial exists to make the gap between them visible.

**Slide 75** Projecting the tutorial problem onto POD modes, at $n_r = 3$

*The point of this derivation is that nothing new happens. Put it next to the spectral projection from Section 3 and let them see that only $\boldsymbol{\Phi}$ changed.*

The lifted problem, on the grid, is

$$\dot{\boldsymbol{v}} = \underbrace{\boldsymbol{h} + \alpha \boldsymbol{f}}_{\text{constant}} + \underbrace{\boldsymbol{v} + \nu \boldsymbol{D}_2 \boldsymbol{v} - \boldsymbol{h} \odot (\boldsymbol{D}_1 \boldsymbol{v})}_{\text{linear}} \underbrace{- \boldsymbol{v} \odot (\boldsymbol{D}_1 \boldsymbol{v})}_{\text{quadratic}}$$

with $\odot$ the entrywise product. Substitute $\boldsymbol{v} = \boldsymbol{\Phi}\boldsymbol{a}$ with $\boldsymbol{\Phi} = [\boldsymbol{\phi}_1, \boldsymbol{\phi}_2, \boldsymbol{\phi}_3]$ and impose $\langle \mathcal{R}, \boldsymbol{\phi}_m \rangle_\Omega = 0$ for $m = 1, 2, 3$, that is $\boldsymbol{\Phi}^\top \boldsymbol{W}_s \mathcal{R} = \boldsymbol{0}$.

**Term 1, the mass matrix.**

$$\boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{\Phi} = \begin{bmatrix} \langle \boldsymbol{\phi}_1, \boldsymbol{\phi}_1 \rangle_\Omega & \langle \boldsymbol{\phi}_1, \boldsymbol{\phi}_2 \rangle_\Omega & \langle \boldsymbol{\phi}_1, \boldsymbol{\phi}_3 \rangle_\Omega \\ \langle \boldsymbol{\phi}_2, \boldsymbol{\phi}_1 \rangle_\Omega & \langle \boldsymbol{\phi}_2, \boldsymbol{\phi}_2 \rangle_\Omega & \langle \boldsymbol{\phi}_2, \boldsymbol{\phi}_3 \rangle_\Omega \\ \langle \boldsymbol{\phi}_3, \boldsymbol{\phi}_1 \rangle_\Omega & \langle \boldsymbol{\phi}_3, \boldsymbol{\phi}_2 \rangle_\Omega & \langle \boldsymbol{\phi}_3, \boldsymbol{\phi}_3 \rangle_\Omega \end{bmatrix} = \boldsymbol{I}_3$$

*This is the one place where the weighting has to be right. If the modes were obtained from an unweighted SVD and then used with a weighted inner product, this matrix is not the identity, and the reduced model quietly solves the wrong problem.*

**Term 2, the constant.** Three numbers, computed once:

$$\boldsymbol{c}_r = \boldsymbol{\Phi}^\top \boldsymbol{W}_s (\boldsymbol{h} + \alpha \boldsymbol{f}) = \underbrace{\boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{h}}_{\boldsymbol{g}_r} + \alpha \underbrace{\boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{f}}_{\boldsymbol{f}_r} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \end{bmatrix} + \alpha \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix}.$$

**Term 3, the linear operator.** A $3 \times 3$ matrix, split by parameter:

$$\boldsymbol{L}_r(\nu) = \underbrace{\boldsymbol{\Phi}^\top \boldsymbol{W}_s \big(\boldsymbol{\Phi} - \boldsymbol{h} \odot (\boldsymbol{D}_1 \boldsymbol{\Phi})\big)}_{\boldsymbol{A}_r} + \nu \underbrace{\boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{D}_2 \boldsymbol{\Phi}}_{\boldsymbol{B}_r}$$

**Term 4, the quadratic tensor.** 27 numbers at $n_r = 3$:

$$C_{mnk} = -\sum_{i=1}^{n_s} w_i \, \phi_m(x_i) \, \phi_n(x_i) \, (\boldsymbol{D}_1 \boldsymbol{\phi}_k)(x_i), \qquad m, n, k \in \{1, 2, 3\}.$$

Written out, the first component of the quadratic term is

$$\big[\boldsymbol{C} : (\boldsymbol{a} \otimes \boldsymbol{a})\big]_1 = C_{111} a_1 a_1 + C_{112} a_1 a_2 + C_{113} a_1 a_3 + C_{121} a_2 a_1 + \cdots + C_{133} a_3 a_3,$$

nine products for one component, 27 in total.

**The result.**

$$\boxed{\dot{\boldsymbol{a}} = \boldsymbol{g}_r + \alpha \boldsymbol{f}_r + (\boldsymbol{A}_r + \nu \boldsymbol{B}_r)\boldsymbol{a} + \boldsymbol{C}_r : (\boldsymbol{a} \otimes \boldsymbol{a}), \qquad \boldsymbol{a}(0) = \boldsymbol{0}}$$

**The affine structure, and why it is the whole game.** Every object above is independent of $\nu$ and $\alpha$. A new operating point therefore costs one scalar multiplication per stored object, and no integral is ever revisited. This is what makes the online stage cheap; it is also what breaks the moment a parameter enters non-affinely, for instance if the source position $x_f$ became a parameter. Then $\boldsymbol{f}_r(x_f)$ would have to be recomputed at every query, and that recomputation touches all $n_s$ grid points, which destroys the independence from $n_s$. Recovering it is exactly what DEIM and ECSW do.

**Slide 76** Counting the online cost

Per time step of the reduced model:

| operation | cost | at $n_r = 8$ |
|---|---|---|
| $\boldsymbol{g}_r + \alpha \boldsymbol{f}_r$ | $O(n_r)$ | 8 flops |
| $\boldsymbol{L}_r \boldsymbol{a}$ | $O(n_r^2)$ | 64 flops |
| $\boldsymbol{C}_r : (\boldsymbol{a} \otimes \boldsymbol{a})$ | $O(n_r^3)$ | 512 flops |
| one solve with $(\boldsymbol{I} - \Delta t \boldsymbol{L}_r)$, prefactored | $O(n_r^2)$ | 64 flops |

Compare with the full-order step, $O(n_s)$ for the explicit finite-difference right-hand side, but with a step $\Delta t < 0.4 \Delta x^2 / \nu$ that shrinks like $n_s^{-2}$.

*Two independent gains, and they should be separated on the board. The state got smaller: $n_r$ instead of $n_s$. And the step got longer, because the stiff operator is now a small matrix that can be inverted implicitly. In the tutorial the second gain is the larger of the two, which surprises people.*

The cubic term is the one that stops scaling. At $n_r = 8$ it is 512 operations and invisible; at $n_r = 50$ it is 125 000 and dominates everything; at $n_r = 200$ the reduced model is slower than a sparse full-order solver. Any statement of the form "the reduced model is $N$ times faster" is meaningless without $n_r$ attached to it.

**Slide 110** Merging bases from several parameter classes

This construction has a section of its own, the next one, because it is the only genuinely new piece of machinery in Tutorial 3 and because the short version of it in an earlier draft of these notes was wrong. See *Merging bases from several classes, slowly*, overleaf.

## 4 Merging bases from several classes, slowly

*This is the one construction in Tutorial 3 that is genuinely new, and it deserves a full board. Everything else is Section 3 with a different* $\boldsymbol{\Phi}$*.*

### What the operation actually is

Two campaigns produced two bases. Each is orthonormal *on its own*:

$$\boldsymbol{\Phi}^{(1)} = [\boldsymbol{\phi}_1^{(1)}, \dots, \boldsymbol{\phi}_q^{(1)}], \qquad \boldsymbol{\Phi}^{(2)} = [\boldsymbol{\phi}_1^{(2)}, \dots, \boldsymbol{\phi}_q^{(2)}], \qquad (\boldsymbol{\Phi}^{(c)})^\top \boldsymbol{W}_s \boldsymbol{\Phi}^{(c)} = \boldsymbol{I}_q.$$

We want one orthonormal basis for $\mathcal{V} = \operatorname{span} \boldsymbol{\Phi}^{(1)} + \operatorname{span} \boldsymbol{\Phi}^{(2)}$, the smallest space containing both.

The obstacle is that the union is *not* orthonormal. Stack it, $\boldsymbol{S} = [\boldsymbol{\Phi}^{(1)}, \boldsymbol{\Phi}^{(2)}] \in \mathbb{R}^{n_s \times 2q}$, and look at its Gram matrix in the $\boldsymbol{W}_s$ inner product:

$$\boldsymbol{S}^\top \boldsymbol{W}_s \boldsymbol{S} = \begin{bmatrix} \boldsymbol{I}_q & \boldsymbol{M} \\ \boldsymbol{M}^\top & \boldsymbol{I}_q \end{bmatrix}, \qquad \boldsymbol{M} = (\boldsymbol{\Phi}^{(1)})^\top \boldsymbol{W}_s \boldsymbol{\Phi}^{(2)}.$$

The diagonal blocks are identities because each class is orthonormal. The off-diagonal block $\boldsymbol{M}$ is everything the two classes have in common, and it is not zero: both regimes solve the same equation, so they share structure.

**The question to put to the room.** $\boldsymbol{S}$ has $2q$ columns. How many *directions* does it contain? Anywhere between $q$ and $2q$, and the answer is not an integer in practice: some directions are shared exactly, some almost. The whole point of what follows is to measure "almost".

**Two quantities, and they are not the same number.** Everything in this section is either an eigenvalue of a Gram matrix or a singular value of the stack, and they differ by a square root. Fix the notation now and keep it:

$$\lambda : \text{eigenvalue of } \boldsymbol{G} = \boldsymbol{S}^\top \boldsymbol{W}_s \boldsymbol{S}, \qquad \sigma : \text{singular value of } \boldsymbol{W}_s^{1/2} \boldsymbol{S}, \qquad \boxed{\sigma = \sqrt{\lambda}}$$

because $(\boldsymbol{W}_s^{1/2}\boldsymbol{S})^\top (\boldsymbol{W}_s^{1/2}\boldsymbol{S}) = \boldsymbol{G}$. Every boxed result below says which of the two it is giving.

### A toy case with one mode per class

Take $q = 1$, two unit vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ at an angle $\theta$, so $\boldsymbol{a}^\top \boldsymbol{b} = \cos\theta$. Then $\boldsymbol{S} = [\boldsymbol{a}, \boldsymbol{b}]$ and

$$\boldsymbol{G} = \boldsymbol{S}^\top \boldsymbol{S} = \begin{bmatrix} 1 & \cos\theta \\ \cos\theta & 1 \end{bmatrix}, \qquad \text{eigenvectors } \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \ \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Substituting each eigenvector back gives the two eigenvalues, and the square root of each gives the corresponding singular value:

$$\boxed{\lambda_\pm = 1 \pm \cos\theta} \quad \text{(eigenvalues of } \boldsymbol{G}) \qquad \Longrightarrow \qquad \boxed{\sigma_\pm = \sqrt{1 \pm \cos\theta}} \quad \text{(singular values)}$$

The corresponding directions are

$$\boldsymbol{u}_1 \propto \boldsymbol{a} + \boldsymbol{b} \quad \text{(what they agree on)}, \qquad \boldsymbol{u}_2 \propto \boldsymbol{a} - \boldsymbol{b} \quad \text{(what tells them apart)}.$$

| $\theta$ | $\lambda_+$ | $\lambda_-$ | $\sigma_+ = \sqrt{\lambda_+}$ | $\sigma_- = \sqrt{\lambda_-}$ | reading |
|---|---|---|---|---|---|
| 0 | 2 | 0 | 1.414 | 0 | the same mode twice; $\boldsymbol{a} - \boldsymbol{b} = \boldsymbol{0}$ |
| 60° | 1.5 | 0.5 | 1.225 | 0.707 | strongly overlapping |
| 90° | 1 | 1 | 1 | 1 | independent; no redundancy |

**The sentence to correct, because the first version of these notes got it backwards.** The columns discarded by the truncation are *not* the directions the classes agree on. The agreement direction is $\boldsymbol{a} + \boldsymbol{b}$, and it has the *largest* singular value: it is kept, and it is the most important direction in the merged space. What is discarded is the *difference* $\boldsymbol{a} - \boldsymbol{b}$ when the two modes nearly coincide. That difference is a direction the stack does not really contain: it is the numerical residue of listing the same physics twice.

### The general statement: principal angles

The toy case is the whole story. For general $q$, let $\boldsymbol{M} = (\boldsymbol{\Phi}^{(1)})^\top \boldsymbol{W}_s \boldsymbol{\Phi}^{(2)}$ have singular values $\cos\theta_1 \geq \cos\theta_2 \geq \cdots \geq \cos\theta_q$. The $\theta_i \in [0, \pi/2]$ are the principal angles between the two subspaces: $\theta_1$ is the smallest angle between any vector of one and any vector of the other, $\theta_2$ the smallest among directions orthogonal to the first pair, and so on.

### From the toy case to $q$ modes, step by step

*The general case is the toy case $q$ times over. The only work is finding the right pair of directions to look at, and that is what the SVD of $\boldsymbol{M}$ does. Take it slowly; there are five steps and none of them is hard.*

**Step 0. What we are computing, and why it is an eigenvalue problem.** The singular values of a matrix $\boldsymbol{X}$ are the square roots of the eigenvalues of $\boldsymbol{X}^\top \boldsymbol{X}$. Here $\boldsymbol{X} = \boldsymbol{W}_s^{1/2} \boldsymbol{S}$, so

$$\boldsymbol{X}^\top \boldsymbol{X} = \boldsymbol{S}^\top \boldsymbol{W}_s^{1/2} \boldsymbol{W}_s^{1/2} \boldsymbol{S} = \boldsymbol{S}^\top \boldsymbol{W}_s \boldsymbol{S} = \boldsymbol{G}, \qquad \boldsymbol{G} = \begin{bmatrix} \boldsymbol{I}_q & \boldsymbol{M} \\ \boldsymbol{M}^\top & \boldsymbol{I}_q \end{bmatrix}.$$

So the whole question is: *what are the eigenvalues of that $2q \times 2q$ matrix?* Nothing else is going on.

**Step 1. Rotate each class internally, and lose nothing.** Take the SVD of the coupling block,

$$\boldsymbol{M} = \boldsymbol{U}\, \boldsymbol{C}\, \boldsymbol{V}^\top, \qquad \boldsymbol{C} = \operatorname{diag}(\cos\theta_1, \ldots, \cos\theta_q),$$

and use it to relabel the modes of each class:

$$\tilde{\boldsymbol{\Phi}}^{(1)} = \boldsymbol{\Phi}^{(1)} \boldsymbol{U}, \qquad \tilde{\boldsymbol{\Phi}}^{(2)} = \boldsymbol{\Phi}^{(2)} \boldsymbol{V}.$$

$\boldsymbol{U}$ and $\boldsymbol{V}$ are $q \times q$ and orthogonal, so each new set is still $\boldsymbol{W}_s$-orthonormal and still spans exactly the same subspace as before. We have changed the *names* of the modes inside each class, not the classes.

**Step 2. In the new names, the coupling is diagonal.** This is the whole trick, and it is one line:

$$(\tilde{\boldsymbol{\Phi}}^{(1)})^\top \boldsymbol{W}_s \tilde{\boldsymbol{\Phi}}^{(2)} = \boldsymbol{U}^\top \underbrace{\left((\boldsymbol{\Phi}^{(1)})^\top \boldsymbol{W}_s \boldsymbol{\Phi}^{(2)}\right)}_{\boldsymbol{M}} \boldsymbol{V} = \boldsymbol{U}^\top (\boldsymbol{U}\boldsymbol{C}\boldsymbol{V}^\top) \boldsymbol{V} = \boldsymbol{C}.$$

Before the rotation, every mode of class 1 overlapped every mode of class 2. After it, mode $i$ of class 1 overlaps *only* mode $i$ of class 2, by the amount $\cos\theta_i$, and is orthogonal to all the others. The two bases have been paired up.

**Step 3. Reorder the columns so partners sit next to each other.** List the $2q$ columns as

$$\tilde{\phi}_1^{(1)},\ \tilde{\phi}_1^{(2)},\ \tilde{\phi}_2^{(1)},\ \tilde{\phi}_2^{(2)},\ \dots$$

instead of all of class 1 then all of class 2. A reordering is a permutation of rows and columns, which never changes eigenvalues. In this order the Gram matrix has entries only on its diagonal and between partners, so it is block diagonal with $q$ blocks of size two:

$$\tilde{\boldsymbol{G}} = \begin{bmatrix} 1 & \cos\theta_1 & & & & \\ \cos\theta_1 & 1 & & & & \\ & & \ddots & & & \\ & & & 1 & \cos\theta_q \\ & & & \cos\theta_q & 1 \end{bmatrix}$$

**Step 4. Each block is the toy case, already solved.** Two paragraphs ago we found that $\left[\begin{smallmatrix} 1 & \cos\theta \\ \cos\theta & 1 \end{smallmatrix}\right]$ has eigenvalues $1 \pm \cos\theta$, with the sum and the difference of the two vectors as eigenvectors. A block-diagonal matrix has, as its eigenvalues, simply the eigenvalues of all its blocks collected together. So

$$\operatorname{eig}(\boldsymbol{G}) = \{\lambda\} = \{\,1+\cos\theta_1, \dots, 1+\cos\theta_q\,\} \cup \{\,1-\cos\theta_1, \dots, 1-\cos\theta_q\,\}.$$

These are *eigenvalues*, not singular values: no square root has been taken yet. That happens in the next step, and only there.

**Step 5. Take the square roots, sort, and read off the pairing.** Now apply $\sigma = \sqrt{\lambda}$ from the reminder above. The angles are ordered $\cos\theta_1 \geq \cdots \geq \cos\theta_q \geq 0$, so the eigenvalues sort as

$$\underbrace{1+\cos\theta_1 \ \geq\ \dots\ \geq\ 1+\cos\theta_q}_{\geq\ 1} \ \geq\ 1\ \geq\ \underbrace{1-\cos\theta_q \ \geq\ \dots\ \geq\ 1-\cos\theta_1}_{\leq\ 1}.$$

Taking the square root of each and counting positions from the top:

$$\boxed{\sigma_i = \sqrt{1+\cos\theta_i}\ \ (i \leq q), \qquad \sigma_{2q+1-i} = \sqrt{1-\cos\theta_i}, \qquad \sigma_i^2 + \sigma_{2q+1-i}^2 = 2}$$

*singular values*, this time. Note where the square root sits: the pair sums to two in square, $\sigma_i^2 + \sigma_{2q+1-i}^2 = 2$, because it is the underlying eigenvalues that add to $(1+\cos\theta_i)+(1-\cos\theta_i) = 2$. The singular values themselves do not add to anything in particular. *Say the last identity in words: the first singular value pairs with the last, the second with the second-to-last, and each pair adds up to two in square. That is a check you can run on any printout in one second, and it is the check performed on the tutorial's numbers on the next page.*

### The same thing with $q = 2$, written out

*If the general argument still feels abstract, do this on the board instead. It is the whole proof at a size you can see.*

Two modes per class. Suppose the coupling block comes out as

$$\boldsymbol{M} = (\boldsymbol{\Phi}^{(1)})^\top \boldsymbol{W}_s \boldsymbol{\Phi}^{(2)} = \begin{bmatrix} 0.6 & 0.6 \\ -0.6 & 0.6 \end{bmatrix}, \qquad \text{so} \qquad \boldsymbol{M} = \boldsymbol{U} \begin{bmatrix} 0.849 & 0 \\ 0 & 0.849 \end{bmatrix} \boldsymbol{V}^\top.$$

Every mode of one class overlaps every mode of the other, and no entry of $\boldsymbol{M}$ tells you by how much the *subspaces* overlap. After Step 1 the coupling becomes $\boldsymbol{C} = \operatorname{diag}(0.849, 0.849)$, and after Step 3 the Gram matrix is

$$\tilde{\boldsymbol{G}} = \left[\begin{array}{cc|cc} 1 & 0.849 & 0 & 0 \\ 0.849 & 1 & 0 & 0 \\ \hline 0 & 0 & 1 & 0.849 \\ 0 & 0 & 0.849 & 1 \end{array}\right], \qquad \{\lambda\} = \{1.849,\ 1.849,\ 0.151,\ 0.151\},$$

$$\{\sigma\} = \{\sqrt{1.849},\ \sqrt{1.849},\ \sqrt{0.151},\ \sqrt{0.151}\} = (1.360,\ 1.360,\ 0.389,\ 0.389), \qquad 1.360^2 + 0.389^2 = 2.00.$$

Both principal angles are $\arccos(0.849) = 32°$: the two subspaces lean on each other equally in both of their directions.

### The numbers from the tutorial

With two classes, $q = 4$, the code prints

$$\Sigma_o = (1.4142,\ 1.4140,\ 1.3587,\ 1.2020,\ 0.7451,\ 0.3923,\ 0.0226,\ 0.0028).$$

Check the pairing, outermost inwards:

| $i$ | $\sigma_i$ | $\sigma_{9-i}$ | $\sigma_i^2 + \sigma_{9-i}^2$ | $\cos\theta_i = \sigma_i^2 - 1$ | $\theta_i$ |
|---|---|---|---|---|---|
| 1 | 1.4142 | 0.0028 | 2.000 | 1.0000 | 0.2° |
| 2 | 1.4140 | 0.0226 | 2.000 | 0.9995 | 1.8° |
| 3 | 1.3587 | 0.3923 | 2.000 | 0.8461 | 32.2° |
| 4 | 1.2020 | 0.7451 | 2.000 | 0.4448 | 63.6° |

*Read the last column out loud, because it is a physical statement about the problem and not about linear algebra. The diffusive and the convective regimes hold two spatial structures in common to within two degrees. A third is shared at thirty-two degrees, a fourth only loosely at sixty-four. Nothing is fully independent: the two regimes are the same equation at different $\nu$, so of course they overlap. That overlap is why a global basis of eight modes does the work of two separate bases of four, and it is also why it does not cost twice as much as it looks.*

### So which columns are actually removed? None.

*This is the question the table invites and does not answer, so answer it head on. The honest reply is that the tutorial removes nothing, and the earlier phrase "effectively six-dimensional" was loose talk. Here is what is really going on.*

**The code's tolerance never fires.** The truncation in step 1 is

```
keep = s_overlap > tol * s_overlap[0]
```

with `tol`$= 10^{-10}$. The threshold is therefore $10^{-10} \times 1.4142 = 1.41 \cdot 10^{-10}$, and the smallest measured overlap value is 0.0028, larger by a factor of $2.5 \cdot 10^7$. *All eight columns survive.* The global basis has eight modes, not six.

> That tolerance is not there to compress anything. It is there to catch *exact* linear dependence, which happens when a class contributes a mode another class already contains to the last bit, or when $q$ is larger than the rank of a class's own data. If such a column were kept, $\boldsymbol{Q}$ would be rank deficient, $\boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{\Phi}$ would be singular, and the reduced model would fail with a linear-algebra error rather than a wrong answer. Compression is the job of step 2, not of this line.

**A small overlap value does not license removal.** $\sigma_o = 0.0028$ says the direction is *nearly* a duplicate. Nearly is not exactly. Dropping it is a modelling decision, and the overlap spectrum alone gives no rule for making it: there is no gap in that list to cut at.

**And the two orderings disagree, so "the last two" is ambiguous.** This is the real trap. After step 1 the columns are ordered by overlap; after step 2 they are reordered by energy, and the two orders are genuinely different. Measured on the tutorial's own data, with the energy of an overlap-ordered column $\boldsymbol{q}_j$ taken as $\|\boldsymbol{q}_j^\top \boldsymbol{W}_s \boldsymbol{D}\|^2$ relative to the total:

| column of $\boldsymbol{Q}$ | $\sigma_o$ *(overlap)* | share of the data energy |
|---:|---:|---:|
| 1 | 1.4142 | $9.14 \cdot 10^{-1}$ |
| 2 | 1.4140 | $7.21 \cdot 10^{-2}$ |
| 3 | 1.3587 | $3.30 \cdot 10^{-3}$ |
| 4 | 1.2020 | $7.40 \cdot 10^{-3}$ |
| 5 | 0.7451 | $2.88 \cdot 10^{-3}$ |
| 6 | 0.3923 | $3.51 \cdot 10^{-4}$ |
| 7 | 0.0226 | $4.62 \cdot 10^{-5}$ |
| 8 | 0.0028 | $8.84 \cdot 10^{-6}$ |

Look at rows 3 and 4: the column with the *larger* overlap carries less than half the energy of the one below it. The two rankings are not the same ranking.

**The rule, stated cleanly.**

- The overlap spectrum *diagnoses*. It says how much the regimes have in common, and it is read once, for understanding.
- The energy spectrum *decides*. If a basis is to be truncated, it is truncated in the energy ordering produced by step 2, never in the overlap ordering of step 1.

Reading a truncation off the overlap table is the mistake this page exists to prevent.

**If you did truncate, here is the number to look at.** The energy spectrum of the finished global basis is

$$\boldsymbol{\Sigma}_z = (10.544,\ 1.580,\ 0.627,\ 0.256,\ 0.214,\ 0.126,\ 0.054,\ 0.022),$$

with cumulative energies

$$(0.9735,\ 0.99541,\ 0.99886,\ 0.99943,\ 0.99983,\ 0.99997,\ 0.999996,\ 1).$$

Cutting from eight to six leaves out $3.0 \cdot 10^{-5}$ of the pooled energy, three thousandths of one percent. By that criterion four modes would already do, since 99.9% is reached at four.

**And yet the tutorial keeps all eight, which is the point of the whole construction.** Pooled energy is dominated by the diffusive class, whose $\sigma_1$ is 9.7 against 4.2. Truncating by pooled energy therefore optimises for the regime that happens to be louder, and the earlier experiment measures what that costs: at the convective probe, a four-mode basis taken from the diffusive class alone gives $2.2 \cdot 10^{-2}$, while the eight-mode global basis gives $6.8 \cdot 10^{-3}$. The four modes the energy criterion would discard are worth a factor of three at the operating point they were collected for.

So a basis built to cover $n_c$ regimes is kept at its full size $n_c q$, and the energy criterion is used *within* a regime, not across regimes. That is the sentence to remember from this section.

## If a value did fall below the tolerance, which vector would go?

*The natural guess is that one of the eight modes is deleted, and that guess is wrong. Nothing that goes into the stack ever comes out of it. What the SVD returns is a different set of eight vectors, and it is one of those that would be dropped.*

**First, keep the two letters straight.** The merging is done on the *spatial* modes $\boldsymbol{\phi}_r$, the columns of $\boldsymbol{\Phi}$. The temporal structures $\boldsymbol{\psi}_r$ play no part in it; they belong to each class's own decomposition and are never stacked.

**What the columns of $\boldsymbol{U}$ actually are.** Recall step 4: in the paired coordinates the Gram matrix splits into blocks $\left[\begin{smallmatrix} 1 & \cos\theta_i \\ \cos\theta_i & 1 \end{smallmatrix}\right]$, whose eigenvectors are $(1,1)/\sqrt{2}$ and $(1,-1)/\sqrt{2}$.

Translated back, those coordinates are the *rotated* modes $\tilde{\boldsymbol{\phi}}_i^{(1)}$ and $\tilde{\boldsymbol{\phi}}_i^{(2)}$, so the left singular vectors of the stack are

$$\boldsymbol{u}_i^{+} = \frac{\tilde{\boldsymbol{\phi}}_i^{(1)} + \tilde{\boldsymbol{\phi}}_i^{(2)}}{\sqrt{2+2\cos\theta_i}} \quad (\sigma = \sqrt{1+\cos\theta_i}), \qquad \boldsymbol{u}_i^{-} = \frac{\tilde{\boldsymbol{\phi}}_i^{(1)} - \tilde{\boldsymbol{\phi}}_i^{(2)}}{\sqrt{2-2\cos\theta_i}} \quad (\sigma = \sqrt{1-\cos\theta_i}).$$

Not modes: sums and differences of paired modes. Checked against the tutorial's own SVD, every one of the eight columns matches one of these to six decimal places:

| column | is | column | is |
|---|---|---|---|
| 1 | $(\tilde{\boldsymbol{\phi}}_1^{(1)} + \tilde{\boldsymbol{\phi}}_1^{(2)})/\lVert\cdot\rVert$ | 8 | $(\tilde{\boldsymbol{\phi}}_1^{(1)} - \tilde{\boldsymbol{\phi}}_1^{(2)})/\lVert\cdot\rVert$ |
| 2 | $(\tilde{\boldsymbol{\phi}}_2^{(1)} + \tilde{\boldsymbol{\phi}}_2^{(2)})/\lVert\cdot\rVert$ | 7 | $(\tilde{\boldsymbol{\phi}}_2^{(1)} - \tilde{\boldsymbol{\phi}}_2^{(2)})/\lVert\cdot\rVert$ |
| 3 | $(\tilde{\boldsymbol{\phi}}_3^{(1)} + \tilde{\boldsymbol{\phi}}_3^{(2)})/\lVert\cdot\rVert$ | 6 | $(\tilde{\boldsymbol{\phi}}_3^{(1)} - \tilde{\boldsymbol{\phi}}_3^{(2)})/\lVert\cdot\rVert$ |
| 4 | $(\tilde{\boldsymbol{\phi}}_4^{(1)} + \tilde{\boldsymbol{\phi}}_4^{(2)})/\lVert\cdot\rVert$ | 5 | $(\tilde{\boldsymbol{\phi}}_4^{(1)} - \tilde{\boldsymbol{\phi}}_4^{(2)})/\lVert\cdot\rVert$ |

So the first-with-last pairing of the singular values is not a coincidence of sorting: column $i$ and column $2q+1-i$ are the sum and the difference of the *same* pair of modes.

**The answer, in one line.**

below tolerance $\implies$ the *difference* $\boldsymbol{u}_i^{-}$ is dropped; the sum $\boldsymbol{u}_i^{+}$ stays

For $\theta_1 = 0.22°$ that would be column 8. Neither $\tilde{\boldsymbol{\phi}}_1^{(1)}$ nor $\tilde{\boldsymbol{\phi}}_1^{(2)}$ is deleted: they are almost the same vector, and the direction they both point in is kept.

**And what does that cost?** Removing $\boldsymbol{u}_i^{-}$ costs each of the two modes the component it had along it, which is exactly half of $\lVert\tilde{\boldsymbol{\phi}}_i^{(1)} - \tilde{\boldsymbol{\phi}}_i^{(2)}\rVert = \sqrt{2-2\cos\theta_i}$. So

$$\text{error on each mode} = \tfrac{1}{2}\sqrt{2-2\cos\theta_i} = \frac{\sigma_i^{-}}{\sqrt{2}} \approx \frac{\theta_i}{2} \quad \text{(radians, for small } \theta\text{)}.$$

Measured on the tutorial by dropping columns 7 and 8 and re-projecting:

| pair | $\theta_i$ | $\theta_i/2$ in radians | measured error |
|---|---|---|---|
| 1 | $0.224°$ | $1.96 \cdot 10^{-3}$ | $1.95 \cdot 10^{-3}$ |
| 2 | $1.835°$ | $1.60 \cdot 10^{-2}$ | $1.60 \cdot 10^{-2}$ |
| 3 | $32.2°$ | *(kept)* | $9 \cdot 10^{-16}$ |
| 4 | $63.6°$ | *(kept)* | $1 \cdot 10^{-15}$ |

*The prediction and the measurement agree to three digits, and pairs 3 and 4 are reproduced to machine precision because their difference directions were kept. That is the sanity check to run if anyone doubts the picture.*

### What the tolerance means as an angle

The test is relative, `s_overlap > tol * s_overlap[0]`, and the largest overlap value is $\sqrt{2}$ whenever any direction is shared. So a pair is dropped when

$$\sqrt{1-\cos\theta_i} \;<\; \text{tol} \cdot \sqrt{2}.$$

For small angles $1-\cos\theta \approx \theta^2/2$, so $\sqrt{1-\cos\theta} \approx \theta/\sqrt{2}$, and the condition collapses to

$$\theta_i \;<\; 2\,\text{tol} \quad \text{(radians)}$$

*The tolerance is a threshold on the principal angle.* With $\text{tol} = 10^{-10}$ the cut is at $2 \cdot 10^{-10}$ radians, which is why nothing is ever dropped: it catches only directions that coincide to machine precision.

| if you want to merge pairs agreeing within | set `tol` to | drops, here |
|---|---|---|
| $10^{-8}$ degrees *(the code's default)* | $10^{-10}$ | nothing |
| $0.5°$ | $4.4 \cdot 10^{-3}$ | pair 1 only |
| $3°$ | $2.6 \cdot 10^{-2}$ | pairs 1 and 2 |
| $45°$ | 0.38 | pairs 1, 2 and 3 |

Raising the tolerance is a legitimate thing to do, and it is how one would build a genuinely smaller merged basis: at tol $= 2.6 \cdot 10^{-2}$ the eight columns become six, with a worst-case representation error of $1.6 \cdot 10^{-2}$ on the modes that were merged. But note what has been bought and what has been paid. The saving is two states out of eight; the price is that the two classes can no longer be told apart in those two directions, and the reduced model will reproduce whichever regime the surviving sum leans towards. On a problem where the regimes are genuinely close, that is a good trade. On one where the whole point is to span both, it is not, and it is better to keep the columns and truncate later by energy, which is what the tutorial does.

### Why an SVD and not a QR

*This is a fair question and the honest answer has two halves. Give both.*

**For orthonormalising alone, QR is fine, and cheaper.** $\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{R}$ gives an orthonormal $\boldsymbol{Q}$ spanning the same space, at roughly a third of the cost of an SVD. If the only requirement were "produce a basis", QR would be the right tool and nothing would be lost.

**But three things are wanted here that QR does not give.**

*1. A symmetric answer.* QR processes columns in order and never revisits them, so the first class passes through untouched and the second gets only what is left over. Running the tutorial's stack through `numpy.linalg.qr` gives

$$|\operatorname{diag} \boldsymbol{R}| = (1,\ 1,\ 1,\ 1,\ 0.129,\ 0.145,\ 0.145,\ 0.022).$$

The first four entries are exactly one: those are the diffusive modes, kept verbatim because they came first. List the convective class first and the answer changes. The SVD treats the two classes symmetrically, which is what "merge" should mean.

*2. A meaningful measure of near-dependence.* Compare the QR diagonal above with $\Sigma_o$. The $\boldsymbol{R}$ diagonal is not ordered, its entries are not the singular values, and it depends on the column order; it bounds the smallest singular value but does not deliver it. Column-pivoted QR improves this and is a reasonable substitute if the SVD is too expensive, but it still gives estimates where the SVD gives the principal angles exactly.

*3. Step two needs an SVD anyway.* Orthonormalising is only half the construction. The merged basis then has to be ranked, and ranking is a POD inside the merged space, which is an SVD by definition. Since one is being computed regardless, using an SVD for the first step too costs almost nothing and yields the diagnostic for free.

Do not confuse the two SVDs. The first, on $\boldsymbol{W}_s^{1/2}\boldsymbol{S}$, is $n_s \times 2q$ and answers *how many independent directions are there*. The second, on $\boldsymbol{Z} = \boldsymbol{Q}^\top \boldsymbol{W}_s \boldsymbol{D}$, is $p \times n_t$ with $p \approx 8$ and answers *which of them matter most in the data*. They measure different things and their singular values are unrelated: the first are overlaps between 0 and $\sqrt{n_c}$, the second are energies.

### Step two, and why the order from step one is useless

After step one the columns of $\boldsymbol{Q}$ are ordered by *overlap*: the directions the classes most agree on come first. That has nothing to do with how much of the data they explain. A structure

both regimes contain may be tiny; a structure only the convective class has may carry most of its energy.

So express the pooled snapshots in the merged space and run a small POD there:

$$\boldsymbol{Z} = \boldsymbol{Q}^\top \boldsymbol{W}_s \boldsymbol{D} \in \mathbb{R}^{p \times n_t}, \qquad \boldsymbol{Z} = \boldsymbol{U}_z \boldsymbol{\Sigma}_z \boldsymbol{V}_z^\top, \qquad \boldsymbol{\Phi}_{\text{glob}} = \boldsymbol{Q}\, \boldsymbol{U}_z.$$

Three things to point out while writing this.

- $\boldsymbol{Z}$ holds the coordinates of every snapshot in the merged basis. Nothing is lost that lies in span $\boldsymbol{Q}$, and what lies outside it is already gone: this step cannot repair a bad step one.
- $\boldsymbol{U}_z$ is $p \times p$ and orthogonal, so $\boldsymbol{Q}\boldsymbol{U}_z$ is still $\boldsymbol{W}_s$-orthonormal. Rotating an orthonormal basis leaves it orthonormal; only the *labels* change.
- $\boldsymbol{Z}$ is a dozen rows by a thousand columns. Its SVD is free compared with one on $\boldsymbol{D}$ itself, which is exactly the trade of the method of snapshots, applied a second time.

Ranking by *pooled* energy favours whichever class carries more of it. Here the diffusive class has $\sigma_1 = 9.7$ against 4.2, so its structures take the leading positions, and truncating the global basis below $2q$ would quietly turn it back into a diffusive basis. Two ways out: normalise each class before pooling, or refuse to truncate below $n_c q$ and accept that covering $n_c$ regimes costs $n_c$ times a single-regime basis. The tutorial takes the second, which is why the global basis there is always used at its full size of eight.

## 5 Reading the Tutorial 3 figures

*Ten figures come out of the solution script. Each one answers a question, and the questions are in a deliberate order. This section says, for each of them, what is on the axes, what the curves are, and the one thing to point at.*

### 1. `t3_singular_values.png` *(two panels)*

**Left panel.** Ordinate: $\sigma_r/\sigma_1$, each spectrum normalised by its own first value, on a log axis. Abscissa: the mode index $r$. Four curves: the diffusive class alone, the convective class alone, both classes pooled into one dataset, and the global basis. A vertical line marks $n_r = 8$, where the global basis runs out of directions.

**What to point at.** The convective spectrum lies *above* the diffusive one everywhere. Same equation, same domain, same source: a lower $\nu$ means a steeper front, and a steeper front needs more modes. The pooled spectrum sits between them, closer to the convective one, because the harder regime dictates the tail. The global basis tracks the pooled curve, which is the statement that merging four modes per class loses very little against a monolithic POD of everything.

**Right panel.** Ordinate: $1 - E_{n_r}$, the *neglected* energy, on a log axis, with the 99% and 99.9% levels marked. Abscissa: the number of modes kept.

> The neglected energy is plotted, not the cumulative energy, and the reason is worth saying. $E_{n_r}$ reaches 0.99 after two or three modes and then looks flat forever: on a linear axis every method appears identical and the plot carries no information. Its complement $1 - E_{n_r}$ falls over ten decades and shows the decay rate, which is the quantity that actually decides $n_r$.

**The numbers.** 99.9% of the energy needs two modes for the diffusive class, five for the convective, four for the pooled data.

### 2. `t3_modes_per_class.png`

Two panels, one per class, each showing $\phi_1$ to $\phi_4$ against $x$ on a shared axis. Same colours in both, so a mode can be compared across regimes.

**What to point at.** The leading modes of the two classes have the same *character* but not the same shape: the convective ones are compressed towards the right, where the front sits. That visual near-agreement is what the principal angles of the previous section quantified as 0.2° and 1.8° for the first two.

### 3. `t3_modes_global.png`

The first four modes of the merged, energy-ranked global basis in colour, drawn over the first four sine modes of Tutorial 2 in grey dashes.

**What to point at.** This is the single most persuasive figure of the tutorial. The sines are spread uniformly across $[0, 1]$, because they were chosen before anyone looked at the problem. The POD modes put all of their structure between $x \approx 0.6$ and $x = 1$, which is where the source sits ($x_f = 0.65$) and where the front forms. *The data-adapted basis knows where the physics is.* That is the whole argument of Section 4 in one picture.

### 4. `t3_temporal.png`

Ordinate: $\psi_r(t)$, the temporal structures, for $r = 1, 2, 3$. Abscissa: time. Colour distinguishes the class, line style distinguishes $r$. Only the first training run of each class is drawn, to keep the figure readable.

**What to point at.** $\psi_1$ moves monotonically and settles: the solution is drifting towards a steady state, and the first mode carries that drift. The higher $\psi_r$ change sign and decay, so

they carry the *transient*. Then note the timescale difference between the two classes, which is the physical statement that a diffusive case settles faster.

> The sign of a POD mode is arbitrary: $\phi_r$ and $\psi_r$ can both be negated without changing anything, since only their product enters. So "rises" and "falls" are not properties of the data, and a student comparing two runs of the same code on different machines may see mirrored curves. What is meaningful is $|\psi_r|$, the relative signs within one decomposition, and the instants at which a curve crosses zero.

### 5. `t3_rom_vs_fom.png`

Ordinate: $u(x, t = 0.5)$. Abscissa: $x$. The thick dark curve is the full-order solution on 198 unknowns; the three thinner curves are POD-Galerkin at $n_r = 2, 4, 8$. The parameter is $\boldsymbol{\mu} = (0.032, 1.2)$, which was never simulated.

**What to point at.** Two modes place the plateau but miss the front, four place the front but not its steepness, eight are indistinguishable from the reference at this scale. Then say the number: eight states against 198.

### 6. `t3_rom_vs_fom.gif` and `t3_frames/`

The same comparison over the whole run, 50 frames at 12 fps, replayed in the deck with `animategraphics`. The running label gives $t$, $\nu$ and $\alpha$.

**What to point at.** Where the $n_r = 2$ curve separates from the reference, and when. It is worst during the transient and recovers as the solution settles, which is the visual form of what `t3_temporal.png` showed: the higher modes exist to carry the transient.

### 7. `t3_two_errors.png`

Ordinate: relative $L^2$ error at $t = 0.5$, log axis. Abscissa: $n_r$ from 2 to 8. **Two curves, and the distinction is the point of the figure:**

| curve | reconstruction | coefficients from |
|---|---|---|
| projection, dashed | $h + \boldsymbol{\Phi}_{n_r} \boldsymbol{a}^{\text{proj}}$ | $\boldsymbol{a}^{\text{proj}} = \boldsymbol{\Phi}_{n_r}^{\top} \boldsymbol{W}_s (\boldsymbol{u} - \boldsymbol{h})$, the known solution |
| POD-Galerkin, solid | $h + \boldsymbol{\Phi}_{n_r} \boldsymbol{a}(t)$ | integrating the reduced system |

The dashed curve needs the full-order answer to compute and is therefore not a model: it is the *best any method could do on this space*. The solid curve is the model. The vertical gap between them is the cost of the dynamics.

**What to point at.** The gap, and that it is not constant. At $n_r = 8$ the two are $9.7 \cdot 10^{-4}$ and $3.3 \cdot 10^{-3}$. A better integrator moves the solid curve towards the dashed one and no further; a better basis moves both.

### 8. `t3_global_vs_local.png`

Two panels, one per probe point: a diffusive one at $\nu = 0.10$ and a convective one at $\nu = 0.01$. In each, three groups of two bars: the three bases, and for each the projection error in grey next to the model error in orange. Log ordinate, shared between panels so the two can be compared by eye. Each basis is used at its *natural* size, four modes for a single class and eight for the merged one, which is written under each label.

**What to point at.** Read across, not down. The diffusive basis is excellent at the diffusive point ($2.4 \cdot 10^{-4}$) and two orders of magnitude worse at the convective one ($2.2 \cdot 10^{-2}$); the convective basis is the mirror image. The global basis is the only column that is acceptable in both panels. *It is not the best anywhere and it is the only one usable everywhere*, and that trade is exactly what the extra four modes buy.

### 9. `t3_cost.png` *(two panels)*

**Left.** Wall clock for one run, log ordinate, in milliseconds: one dark bar for the full-order model at 198 unknowns, then one orange bar per $n_r$. The point is the height ratio, about 30, and that the orange bars are all the same height.

**Right.** Twin axes against $n_r$: the relative error in orange on the left axis, log, and the speed-up in dark on the right axis, linear.

**Why the orange bars are all the same height, which is the question this figure raises and does not answer.** Because at $n_r \leq 8$ the wall clock is the Python loop, not the arithmetic. And because almost all of the speed-up is not the smaller state at all: the full-order run takes 3168 explicit steps at $\Delta t = 3.16 \cdot 10^{-4}$, the reduced one 100 at $10^{-2}$, a ratio of 31.7 against a measured speed-up of about 32. The reduced model is faster because it can take a longer step, and it can take a longer step because its stiff operator is an $8 \times 8$ matrix that is factorised once.

### 10. `t3_online_scaling.png`

Ordinate: time for one evaluation of a single term, in microseconds, log axis. Abscissa: $n_r$ from 4 to 96, log axis. Two measured curves, the linear term $\boldsymbol{L}_r\boldsymbol{a}$ and the quadratic contraction $\boldsymbol{C}_r : (\boldsymbol{a} \otimes \boldsymbol{a})$, and two grey reference slopes $n_r^2$ and $n_r^3$ anchored at the *last* point.

**What to point at.** Below $n_r \approx 30$ both curves are flat. That plateau is not physics: it is the fixed cost of calling into `numpy`, and it is why the cost figure showed no dependence on $n_r$. Above it the quadratic term takes off, from $42\,\mu$s at $n_r = 4$ to $255\,\mu$s at $n_r = 96$, with a fitted exponent of 1.8 still climbing towards 3 as the overhead is washed out. *That exponent is why hyper-reduction exists*: at $n_r \approx 200$ one reduced step would cost as much as one full-order step, and the whole enterprise collapses.

## 6 Answers to the "Over to you" boxes

*Seventeen questions are put to the room across the day, in the blue boxes. They are meant to be worked at the board, not read out, so what follows is the answer plus the thing to draw out of the audience on the way to it. In slide order.*

### Section 2: approximation and projection

**Slide 8** Shapes in the chain rule, and the order

$J = \boldsymbol{e}^\top \boldsymbol{e}$ is a scalar, $\boldsymbol{e} = \boldsymbol{u} - \boldsymbol{\Phi c} \in \mathbb{R}^{n_x}$, and $\boldsymbol{c} \in \mathbb{R}^{n_b}$.

| object | shape | value |
|---|---|---|
| $\partial J/\partial \boldsymbol{e}$ | $1 \times n_x$ *(a row)* | $2\boldsymbol{e}^\top$ |
| $\partial \boldsymbol{e}/\partial \boldsymbol{c}$ | $n_x \times n_b$ *(a Jacobian)* | $-\boldsymbol{\Phi}$ |
| $\partial J/\partial \boldsymbol{c}$ | $1 \times n_b$ | $-2\boldsymbol{e}^\top \boldsymbol{\Phi}$ |

*The order is forced by the shapes: $(1\times n_x)(n_x \times n_b)$ is the only product that exists. Outer derivative first, inner second. Students who write it the other way get a shape error, which is the fastest possible feedback.*

Setting it to zero gives $\boldsymbol{\Phi}^\top \boldsymbol{e} = \boldsymbol{0}$, the orthogonality condition, and substituting $\boldsymbol{e}$ gives $\boldsymbol{\Phi}^\top \boldsymbol{\Phi}\, \boldsymbol{c} = \boldsymbol{\Phi}^\top \boldsymbol{u}$: the normal equations were not postulated, they are stationarity.

**Slide 14** Monomials against Legendre on the same space

**The same approximation error, and wildly different conditioning.** The two families span the identical space of polynomials of degree $\leq n_b - 1$, and the best approximation depends only on the *space*, so in exact arithmetic the two give the same $u_{n_b}$ to the last digit.

What differs is $\boldsymbol{A}$. For Legendre on $(-1, 1)$ it is diagonal, so $\mathrm{cond}(\boldsymbol{A}) = O(1)$. For monomials it is the Hilbert-type matrix, whose condition number grows roughly like $e^{3.5 n_b}$, and Tutorial 1 measures it crossing $10^{16}$ near $n_b = 22$. *So the trap is that both answers are "correct" on paper and only one survives floating point. Conditioning is a property of the basis, accuracy a property of the space, and this question separates them in one line.*

**Slide 17** Where did the derivation use one dimension?

**Nowhere.** Read it back: it used a vector space, an inner product, the definition of the error, and the imposition of orthogonality. Not the dimension of $\Omega$, not that $\Omega$ is connected, not that $u$ is scalar.

So the same three lines give:

- $\Omega \subset \mathbb{R}^d$ for any $d$, with $\mathrm{d}\Omega$ the volume element;
- vector fields, by stacking the components inside each $\phi_n$ and inside $u$;
- complex fields, with a conjugate in the first slot;
- and scattered data, where there is no grid at all, provided a quadrature rule with weights $w_j$ is available.

*The last one is worth thirty seconds. Projection needs points and weights, never connectivity. That is exactly why POD can be computed on a point cloud from particle tracking, with no mesh anywhere.*

**Slide 24** Measuring the decay rate without the exact solution

Compute the coefficients you can, then let the shape of the curve tell you the rate. Plot $|c_n|$ against $n$ twice: on **log-log**, an algebraic decay $c_n \sim n^{-p}$ is a straight line of slope $-p$; on

**semilog**, an exponential decay $c_n \sim e^{-an}$ is. Whichever is straight names the family, and its slope gives the rate.

> Fit the *middle* of the range, never the two ends. The first few coefficients are not yet in the asymptotic regime, and the last ones sit on a plateau set by quadrature error and by $\mathrm{cond}(\boldsymbol{A})$, not by the function. Reading the rate off the plateau is the classic error, and it always reports convergence stopping earlier than it does.

*Add the practical consequence: the plateau height is a measurement of your own numerical noise floor, and it is free. It is the same plateau that appears in Tutorial 2, where it turns out to be the accuracy of the reference solution rather than of the method.*

## Section 3: Galerkin projection

### Slide 45 Propose a rule that turns the residual into $n_b$ equations

Anything that extracts $n_b$ numbers from $\mathcal{R}$ is admissible, and the room usually produces most of these:

| proposal | condition | name |
|---|---|---|
| force it to vanish at $n_b$ points | $\mathcal{R}(x_m) = 0$ | collocation |
| average over $n_b$ cells | $\int_{\Omega_m} \mathcal{R}\,\mathrm{d}\Omega = 0$ | finite volume |
| kill its first $n_b$ moments | $\int \mathcal{R}\, x^m\,\mathrm{d}\Omega = 0$ | method of moments |
| minimise its norm | $\min \lVert\mathcal{R}\rVert_{L^2}^2$ | least squares |
| make it $\perp$ to the trial space | $\langle \mathcal{R}, \phi_m \rangle_\Omega = 0$ | Galerkin |

*Accept all of them, then ask the second question, which is the real one: on what grounds would you choose? Three criteria are worth writing up. Does the resulting system inherit the structure of the operator, its symmetry and its energy balance? Is it well conditioned? Is it cheap? Collocation is cheapest and inherits nothing; least squares always gives a symmetric positive system but squares the condition number; Galerkin inherits the structure, and that is what the next three slides are about.*

### Slide 48 What makes $\boldsymbol{K}$ diagonal as well

$\boldsymbol{A}$ is diagonal as soon as the $\phi_n$ are orthogonal. $\boldsymbol{K}$ is a different matter:

$$K_{mn} = \langle \phi_m, \mathcal{L}\phi_n \rangle_\Omega \text{ diagonal} \iff \mathcal{L}\phi_n \perp \phi_m \text{ for all } m \neq n.$$

The clean sufficient condition is that the $\phi_n$ are eigenfunctions of $\mathcal{L}$, $\mathcal{L}\phi_n = \lambda_n \phi_n$, on top of being orthogonal: then $K_{mn} = \lambda_n \langle \phi_m, \phi_n \rangle_\Omega = \lambda_n \delta_{mn}$. *Two things to add. First, "eigenfunctions of $\mathcal{L}$" means with the boundary conditions attached: $\sin(n\pi x)$ diagonalises $\partial_{xx}$ on $(0,1)$ with homogeneous Dirichlet data and would not with Neumann. Second, this is exactly why the sines diagonalise the diffusion term of the tutorial and do nothing at all for the convection term, which they are not eigenfunctions of. Diagonalising the whole operator is a luxury of linear, constant-coefficient, separable problems.*

### Slide 55 Petrov–Galerkin and the loss of symmetry

$K_{mn} = \langle \psi_m, \mathcal{L}\phi_n \rangle_\Omega$ is in general not symmetric, even when $\mathcal{L}$ is self-adjoint. The proof that Galerkin inherits symmetry used $\langle \phi_m, \mathcal{L}\phi_n \rangle_\Omega = \langle \mathcal{L}\phi_m, \phi_n \rangle_\Omega$, and that step needs $\phi_m$ to be admissible as a *test* function. With $\psi_m \neq \phi_m$ it is not, and the argument stops there.

What is lost, and what is bought:

| lost | bought |
|---|---|
| symmetry of $\boldsymbol{K}$, hence Cholesky | freedom to bias the test space upwind |
| the energy-norm optimality (Cea) | stability at high Péclet number (SUPG) |
| the discrete energy balance | control of the *discrete* residual (LSPG) |

One caveat worth stating, because it looks like a contradiction. Least-squares Petrov–Galerkin *does* give a symmetric positive system, since it forms $\boldsymbol{J}^\top \boldsymbol{J}$. But that is the symmetry of a different operator, the normal equations of the residual, and it is paid for by squaring the condition number. Symmetry was not recovered; a new problem was solved instead.

**Slide 67** $A^e_{11}$, and why $\boldsymbol{K}^e$ is singular

$$A^e_{11} = \int_0^1 (1-\xi)^2 \,\Delta x \,\mathrm{d}\xi = \Delta x \left[ -\frac{(1-\xi)^3}{3} \right]_0^1 = \Delta x \left( 0 + \frac{1}{3} \right) = \frac{\Delta x}{3}.$$

Consistent with $\boldsymbol{A}^e = \frac{\Delta x}{6}\left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$, whose $(1,1)$ entry is $2\Delta x/6 = \Delta x/3$.

**Why $\boldsymbol{K}^e$ is singular.** Its rows sum to zero:

$$\boldsymbol{K}^e \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\Delta x} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

The constant function is in its null space, which is the algebra saying the obvious thing: $\boldsymbol{K}$ is built from $\int \phi'\psi'$, and a constant has zero derivative, so it costs no stiffness energy. *Then the useful follow-up: the assembled $\boldsymbol{K}$ is singular for the same reason, and stays singular until the boundary conditions remove the constant. A pure Neumann problem never removes it, which is why such problems are solvable only up to an additive constant and why the linear solver complains. That is not a bug in the assembly.*

**Slide 72** The diffusive step limit, in numbers

On $n_x = 200$ points of $(0,1)$, $\Delta x = 1/199 = 5.025 \cdot 10^{-3}$, so $\Delta x^2 = 2.525 \cdot 10^{-5}$ and, with the safety factor 0.4 used in the tutorial,

$$\Delta t = \frac{0.4\,\Delta x^2}{\nu} = \frac{0.4 \times 2.525 \cdot 10^{-5}}{0.03} = 3.37 \cdot 10^{-4}, \qquad \frac{T}{\Delta t} = \frac{1}{3.37 \cdot 10^{-4}} \approx 2970 \text{ steps.}$$

which is exactly what the tutorial prints. *Then make them feel the exponent. Three thousand steps for one second of a one-dimensional problem on two hundred points. Double the resolution and it is twelve thousand, because $\Delta t$ falls as $\Delta x^2$ while the number of points only doubles: refining costs a factor of eight in total work, not two. That is the wall the reduced model climbs over in Section 4, and it climbs over it by changing $\Delta t$, not by changing the number of unknowns.*

**Slide 73** Why the nonlinear term is not treated implicitly too

Because the step would stop being a linear solve. Taking $\boldsymbol{n}$ at $n+1$ gives

$$(\boldsymbol{I} - \Delta t\, \boldsymbol{L})\boldsymbol{a}^{n+1} - \Delta t\, \boldsymbol{n}(\boldsymbol{a}^{n+1}) = \boldsymbol{a}^n,$$

which is a *nonlinear* system in $\boldsymbol{a}^{n+1}$, to be solved at every step, typically by Newton. That needs the Jacobian of the quadratic term,

$$\frac{\partial}{\partial \boldsymbol{a}} \left[ \boldsymbol{C} : (\boldsymbol{a} \otimes \boldsymbol{a}) \right]_m = \sum_k (C_{mnk} + C_{mkn}) a_k,$$

which depends on $\boldsymbol{a}$, so the iteration matrix changes at every Newton step of every time step. The single factorisation done once before the loop is gone, replaced by one $O(n^3)$ factorisation per iteration per step.

| | factorisations for the whole run | step limit |
|---|---|---|
| semi-implicit *(as coded)* | 1 | convective only |
| fully implicit | $\sim$ 2–3 per step | none |

*So the trade is stated cleanly: fully implicit removes the last step restriction and costs a few thousand factorisations instead of one. Worth it when convection is the stiff part, which happens at high Reynolds number. Not worth it here, where diffusion was the problem and taking only that implicitly already bought a factor of thirty in step size.*

## Section 4: POD and POD-Galerkin

### Slide 87 Which two-point quantity appears

Write the square as a product of two integrals, in different dummy variables, and take the expectation inside:

$$E\Big[\,|\,\langle u,\phi\rangle_\Omega\,|^2\Big] = E\left[\int_\Omega u(x,\xi)\phi(x)\,\mathrm{d}x \int_\Omega u(x',\xi)\phi(x')\,\mathrm{d}x'\right]$$
$$= \int_\Omega\int_\Omega \phi(x)\,\underbrace{E\{u(x,\xi)\,u(x',\xi)\}}_{C(x,x')}\,\phi(x')\,\mathrm{d}x\,\mathrm{d}x'.$$

The two-point quantity is the covariance kernel $C(x,x')$. *Two things to draw out. First, the exchange of $E$ and the integrals is the only analytical step, and it is legitimate for square-integrable fields. Second, and this is the conceptual point: a one-point field has produced a two-point object. That is not an accident of the algebra. Correlation between two locations is precisely what "coherent structure" means, and it is the only information POD ever uses.*

### Slide 89 What $\boldsymbol{D}$ looks like for a steady solution

Every column is the same vector: $\boldsymbol{D} = \boldsymbol{d}\,\mathbf{1}^\top$.

- rank one, so exactly one nonzero singular value, $\sigma_1 = \|\boldsymbol{d}\|\sqrt{n_t}$;
- $\boldsymbol{\phi}_1 = \boldsymbol{d}/\|\boldsymbol{d}\|$ and $\boldsymbol{\psi}_1 = \mathbf{1}/\sqrt{n_t}$: the single mode is the solution itself, and its time coefficient is constant;
- every other $\sigma_r = 0$, and $E_1 = 1$ exactly.

*The lesson is about sampling, not about physics. POD reports the rank of what it was given. Show the near-version of the same trap: a run that reaches steady state at $t = 0.3$ and is then sampled uniformly to $t = 10$ has most of its columns nearly identical, the leading mode is essentially the mean, and the spectrum looks far more compressible than the dynamics really is. Sampling design is part of the method.*

### Slide 95 What $\sum_{r>n_r}\sigma_r^2$ tells you before any model

It is the floor. Computed from the SVD alone, with no reduced solver in existence, it is the smallest squared error any $n_r$-dimensional *linear* space can leave on this data.

Three uses, all available before writing a line of the model:

1. **Choose $n_r$.** Read off the smallest $n_r$ whose tail is below the tolerance.
2. **Decide whether to continue.** If the tail is still above tolerance at the largest $n_r$ you can afford, stop. No projection method on this space will meet the target, and no closure model will rescue it.
3. **Diagnose later.** It is the dashed curve of Tutorial 3, against which the model error is measured.

*And the warning that belongs with it: the tail bounds the representation error, not the error of the reduced dynamics. The tutorial shows the two differing by a factor of* 170 *at one operating point. A small tail is necessary and not sufficient.*

**Slide 101** Which Tutorial 2 operators survive the change of basis

**None of the projected ones, and all of the full-order ones.** Every reduced object is built by sandwiching something between copies of $\boldsymbol{\Phi}$, and $\boldsymbol{\Phi}$ is precisely what changed.

| object | contains $\boldsymbol{\Phi}$? | verdict |
|---|---|---|
| the grid, $\boldsymbol{W}_s$, $\boldsymbol{D}_1$, $\boldsymbol{D}_2$ | no | reused unchanged |
| the lifting $\boldsymbol{h}$, the source $\boldsymbol{f}$ | no | reused unchanged |
| $\boldsymbol{g}_r$, $\boldsymbol{f}_r$ | yes | rebuilt |
| $\boldsymbol{A}_r$, $\boldsymbol{B}_r$ | yes | rebuilt |
| $\boldsymbol{C}_r$ | yes | rebuilt |

*So the honest answer is that nothing is reused* numerically *and everything is reused* structurally*: the derivation, the assembly routine and the time integrator are the same lines of code, called with a different* $\boldsymbol{\Phi}$*. That is the whole claim of Section 4 in one table, and it is worth letting the disappointment land first: students expect some matrix to survive, and none does.*

**Slide 106** The two quantities that fill the diagnosis table

$$\varepsilon_{\text{proj}} = \frac{\|\boldsymbol{u} - \boldsymbol{h} - \boldsymbol{\Phi}_{n_r}\boldsymbol{\Phi}_{n_r}^{\top}\boldsymbol{W}_s(\boldsymbol{u} - \boldsymbol{h})\|}{\|\boldsymbol{u}\|}, \qquad e_{\text{ROM}} = \frac{\|\boldsymbol{u} - (\boldsymbol{h} + \boldsymbol{\Phi}_{n_r}\boldsymbol{a}(t))\|}{\|\boldsymbol{u}\|}$$

**The first needs no reduced model at all.** It takes the full-order solution, which is known whenever the diagnosis is being run, and projects it. No time integration appears in it anywhere, which is exactly why it is a property of the *space*. The second requires integrating the reduced system, and is therefore a property of the space *and* the dynamics. *The point to land: the cheap one is the informative one. If* $\varepsilon_{\text{proj}}$ *is already too large you can stop without ever writing the model. And the gap between them, not either number alone, is what says which of the two things to go and fix.*

**Slide 115** Is the union of two orthonormal sets orthonormal?

**No.** Each set is orthonormal within itself, so the diagonal blocks of the Gram matrix are identities, but the cross block is not zero:

$$\boldsymbol{S}^{\top}\boldsymbol{W}_s\boldsymbol{S} = \begin{bmatrix} \boldsymbol{I}_q & \boldsymbol{M} \\ \boldsymbol{M}^{\top} & \boldsymbol{I}_q \end{bmatrix}, \qquad \boldsymbol{M} = (\boldsymbol{\Phi}^{(1)})^{\top}\boldsymbol{W}_s\boldsymbol{\Phi}^{(2)} \neq \boldsymbol{0},$$

and it is not zero for a reason: both classes are solutions of the same equation, so they share structure.

**What goes wrong if it is used anyway.** The reduced mass matrix $\boldsymbol{\Phi}^{\top}\boldsymbol{W}_s\boldsymbol{\Phi}$ is no longer $\boldsymbol{I}$. A code that assumes it is silently solves a different problem; a code that does not must invert it at every step. Worse, if the union happens to be rank deficient the mass matrix is singular and the run fails outright.

**What has to be done.** Two steps, and both are needed: orthonormalise, by a weighted SVD of the stack, and then re-rank, because the order that comes out of step 1 is by overlap between the classes and has nothing to do with how much data each direction explains. *This is the question that opens the merging construction, so do not answer it fully here. Get "no, and here is why not" out of the room, then let the next two slides do the work.*

**Slide 122** Which stored object breaks if $x_f$ becomes a parameter

**Exactly one:** $\boldsymbol{f}_r = \boldsymbol{\Phi}^{\top}\boldsymbol{W}_s\boldsymbol{f}$**.** Go through them:

| object | depends on $x_f$? | |
|---|---|---|
| $\boldsymbol{g}_r = \boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{h}$ | no, $h = 1 - x$ | survives |
| $\boldsymbol{f}_r = \boldsymbol{\Phi}^\top \boldsymbol{W}_s \boldsymbol{f}$ | yes, $f$ is centred on $x_f$ | breaks |
| $\boldsymbol{A}_r,\ \boldsymbol{B}_r$ | no, built from $\boldsymbol{\Phi}$, $h$, $\boldsymbol{D}_1$, $\boldsymbol{D}_2$ | survive |
| $\boldsymbol{C}_r$ | no, built from $\boldsymbol{\Phi}$ alone | survives |

**And why one is enough to spoil it.** Rebuilding $\boldsymbol{f}_r$ means evaluating $f$ at all $n_s$ grid points and projecting, at *every* query. The online cost stops being independent of $n_s$, which was the entire point of the affine structure. A model that is small but still touches the fine grid is not fast.

Two remedies, and they are the two halves of the last part of Section 4. *Interpolate the operator*: build $\boldsymbol{f}_r$ at a handful of values of $x_f$ offline and interpolate between them, which works because $\boldsymbol{f}_r$ depends smoothly on $x_f$. Or *interpolate the function*: evaluate $f$ at $m \ll n_s$ selected points and reconstruct it in a second POD basis, which is DEIM.

There is also a quieter consequence, worth raising if the room is quick. If $x_f$ is a parameter then solutions with the source in different places are part of the family, so the *training set* has to cover it too, and the basis will need more modes. The affine structure is not the only thing that breaks; the Kolmogorov width of the solution set grows as well, and a moving source is precisely the translating-feature case that linear reduction handles badly.

## 7 Answers to the tutorial questions

*Every question printed at the end of a tutorial is answered here, with the numbers the solution scripts actually produce. Quote the numbers: a student who has run the code can check them, and one who has not can see what the exercise is worth.*

### Tutorial 1

**Which families are orthogonal, continuously and discretely?** Fourier on a uniform grid and Chebyshev at the CGL points are orthogonal in both senses, at the level of $10^{-16}$. Chebyshev on a *uniform* grid is orthogonal continuously ($10^{-2}$) but not discretely ($10^{-1}$): the quadrature cannot integrate $\phi_m \phi_n w$ exactly, and the singular weight $(1-x^2)^{-1/2}$ near the ends makes it worse. Monomials are orthogonal in neither sense, at $3 \cdot 10^{-1}$. *Settings 3 and 4 use the same functions.* The difference is entirely the placement of the points, and it moves the largest off-diagonal entry of $\boldsymbol{A}$ from $10^{-1}$ to $10^{-16}$.

**At which $n_b$ does the monomial system become useless?** $\mathrm{cond}(\boldsymbol{A})$ crosses $10^{16}$ near $n_b \approx 22$, so the computed coefficients have no correct digits beyond that.

**Does the approximation degrade at the same $n_b$?** Not quite; it keeps improving nearly to that point and only then turns around. *This is worth a minute at the board. Losing digits in $\boldsymbol{c}$ is not the same as losing accuracy in $u_{n_b} = \sum_n c_n \phi_n$. The errors in the coefficients are strongly correlated and largely cancel when the sum is formed. The quantity that is ill-conditioned is the* representation*, not the function it represents.*

**Which target sets the achievable accuracy?** The one with the kink. No basis of smooth global functions represents $|x|$ well, and all four curves fall off algebraically. On the smooth target the three polynomial settings converge fast while Fourier stalls near $2 \cdot 10^{-1}$, because $u_{\mathrm{smooth}}$ is not periodic on $[-1, 1]$: the series is faithfully reconstructing a function that has a jump at the ends.

### Tutorial 2

**How many sine modes match the finite-difference reference?** Sixteen. The measured errors are $6.7 \cdot 10^{-2}$ at $n_b = 2$, $1.1 \cdot 10^{-3}$ at 8, $7.4 \cdot 10^{-5}$ at 16 and $3.3 \cdot 10^{-5}$ at 32. The last step buys almost nothing, because at that point the error is no longer the spectral method's: it is the accuracy of the second-order reference itself.

**Which curve is straight on which axis?** The finite-element curve is straight on log-log, because algebraic convergence $e \sim \Delta x^p$ is a straight line there with slope $p$. The spectral curve is straight on a *semilog* axis, because exponential convergence $e \sim e^{-c n_b}$ is. *Ask them to read the consequence rather than the shape: to gain a digit with finite elements you multiply the mesh; to gain a digit with a spectral basis you add a fixed number of modes. That is the whole argument for global bases on smooth problems, and it is why the POD basis of Tutorial 3 is worth building.*

**Which lines changed between parts B and C?** Only the construction of $\boldsymbol{\Phi}$ and $\boldsymbol{\Phi}'$ at the quadrature points. The routine `assemble` is called with different arguments and is not modified; so are the integrators. *This is the punchline of the whole tutorial: three methods, one derivation, and the code proves it.*

**Replacing the semi-implicit solver by RK4 in part C.** $\Delta t$ must fall to the diffusive stability limit of the finite-element operator, which scales like $\Delta x^2 / \nu$. With 50 elements and $\nu = 0.03$ that is around $5 \cdot 10^{-3}$ times a safety factor, against the $2 \cdot 10^{-3}$ used for accuracy, so the penalty is mild here. Refine the mesh once and it is not: the limit falls by four while the accuracy requirement falls by two.

## Tutorial 3

**1. Which error does a better integrator improve?** Only $e_{\mathrm{ROM}}$, and only the part of it above $\varepsilon_{\mathrm{proj}}$. The projection error is $\varepsilon_{\mathrm{proj}} = \|u - h - \boldsymbol{\Phi}_{n_r}\boldsymbol{\Phi}_{n_r}^{\top}\boldsymbol{W}_s(u-h)\|/\|u\|$, computed from the *known* full-order solution, with no time integration anywhere in it. Nothing about the solver can move it. A larger or better basis moves both, because $\varepsilon_{\mathrm{proj}}$ is the floor under $e_{\mathrm{ROM}}$.

At the test point, $\boldsymbol{\mu} = (0.032, 1.2)$ with $n_r = 8$: $\varepsilon_{\mathrm{proj}} = 9.7 \cdot 10^{-4}$ against $e_{\mathrm{ROM}} = 3.3 \cdot 10^{-3}$, a factor 3.4. At the diffusive probe the same basis gives $9.3 \cdot 10^{-6}$ against $1.6 \cdot 10^{-3}$, a factor 170. *So inside the training set the space is excellent and the* dynamics *is what limits the model. That is the opposite of what most people guess, and it is why the diagnosis has to be measured rather than assumed.*

**2. What does the overlap spectrum mean?** Stack $q$ orthonormal modes from each of $n_c$ classes and take the singular values of the stack. A direction that *every* class contains appears $n_c$ times, and the stacked matrix has the singular value $\sqrt{n_c}$ there. A direction only one class has appears once, giving 1. A direction two classes hold in *nearly* the same shape gives one large value and one small one: the small value belongs to the difference of two almost identical modes and carries no new information.

Measured, with $n_c = 2$ and $q = 4$:

$$\Sigma_o = (1.414,\ 1.414,\ 1.359,\ 1.202,\ 0.745,\ 0.392,\ 0.023,\ 0.003).$$

Two directions are shared exactly, two nearly so, two belong to one class, and the last two are near-duplicates. Note that none of them is *removed*: the code's tolerance is $10^{-10}$ and the smallest value here is 0.0028, larger by seven orders of magnitude, so the global basis keeps all eight. The overlap spectrum diagnoses redundancy; it does not truncate anything.

| the overlap spectrum measures how much physics the regimes have in common |
|---|

With three classes the shared value would be $\sqrt{3} \approx 1.732$.

**3. Where does the speed-up stop growing?** It never grows in the measured range: the speed-up is between 28 and 38 for every $n_r$ from 2 to 8, and flat. The reason is that the wall clock is spent in the Python loop, not in arithmetic: at $n_r = 8$ the quadratic contraction is $8^3 = 512$ floating-point operations, and the measurement shows 44 $\mu$s per call, almost all of it the cost of entering `einsum`.

The scaling test separates the two. Below $n_r \approx 30$ both terms are flat; above it the quadratic term rises, reaching 255 $\mu$s at $n_r = 96$ against 42 $\mu$s at $n_r = 4$, with a fitted exponent of 1.8 still climbing towards 3. Extrapolating, one reduced step would match one full-order run near $n_r \approx 200$, which for this problem is $n_s$: at that point the reduced model has no reason to exist.

**4. Where the speed-up actually comes from, and it is not what they expect.** Count the steps, not the unknowns.

| | $\Delta t$ | steps to $t = 1$ |
|---|---|---|
| full order, explicit, $\nu = 0.032$ | $3.16 \cdot 10^{-4}$ | 3168 |
| reduced, semi-implicit | $10^{-2}$ | 100 |

The ratio of step counts alone is 31.7, and the measured speed-up is about 32. *Essentially all of it is the longer time step*, not the smaller state. The state got smaller too, but at $n_r = 8$ that saving is buried under interpreter overhead. The reason the step can grow is that the stiff operator is now an $n_r \times n_r$ matrix that can be factorised once and treated implicitly, which was impractical at $n_s = 198$ inside an explicit code.

**4. Extrapolating to $\nu = 0.005$: space or dynamics?** The space. Measured at $n_r = 8$, at $t = 0.5$:

| $\nu$ | $\varepsilon_{\text{proj}}$ | $e_{\text{ROM}}$ | ratio |
|---|---|---|---|
| 0.032 *(inside)* | $9.7 \cdot 10^{-4}$ | $3.3 \cdot 10^{-3}$ | 3.4 |
| 0.010 *(on the edge)* | $6.8 \cdot 10^{-3}$ | $2.0 \cdot 10^{-2}$ | 2.9 |
| 0.005 *(outside)* | $1.6 \cdot 10^{-2}$ | $2.7 \cdot 10^{-2}$ | 1.6 |

The projection error grows by a factor 17 from the first row to the last, while the ratio $e_{\text{ROM}}/\varepsilon_{\text{proj}}$ *falls*. The dynamics is not getting worse; the basis simply cannot represent the steeper front that $\nu = 0.005$ produces, because nothing that steep was in the training set. *That is the diagnostic to teach. If both grow together and the ratio holds, suspect the dynamics. If the projection error grows and the ratio shrinks, the space has run out, and no closure model or better integrator will save it. Only new snapshots will.*

**5. The offline economics at realistic cost.** Here the offline stage is 2.3 s for six training runs, and break-even comes after 11 online queries. *Ask why* 11 *and not* 6. *The six training runs are not all the same price: the run at* $\nu = 0.10$ *needs* 9900 *explicit steps, against* 3168 *for the test point and* 990 *at* $\nu = 0.01$. *The offline bill is set by the* most expensive *corner of the parameter box, not by an average one.*

Scaled to a full-order run of one hour, and keeping the same speed-up $S$:

$$T_{\text{off}} \approx 6 \text{ hours}, \qquad T_{\text{on}} = \frac{3600}{32} \approx 113 \text{ s}, \qquad N_{\text{break-even}} = \frac{T_{\text{off}}}{T_{\text{FOM}} - T_{\text{on}}} \approx \frac{6 \cdot 3600}{3487} \approx 6.$$

$$\boxed{N_{\text{break-even}} \approx \frac{N_{\text{train}}}{1 - 1/S} \xrightarrow[S \gg 1]{} N_{\text{train}}}$$

So with a large speed-up the model pays for itself after roughly as many queries as it took training runs to build, which is the number worth carrying away. A digital twin that answers thousands of queries is far past that point; one that answers five is not worth building.

## 8 Two derivations worth keeping in reserve

### Cea's lemma, in four lines

Useful if someone asks why the residual condition is enough. Assume $\mathcal{L}$ is symmetric and coercive, and write $a(u,w) = \langle \mathcal{L}u, w\rangle_\Omega$ for the associated energy inner product, with norm $\|u\|_a^2 = a(u,u)$.

The exact solution satisfies $a(u,w) = \langle f, w\rangle_\Omega$ for all admissible $w$; the Galerkin solution satisfies $a(u_{n_b}, w) = \langle f, w\rangle_\Omega$ for all $w \in \mathcal{V}$. Subtracting,

$$a(u - u_{n_b}, w) = 0 \qquad \text{for all } w \in \mathcal{V} \qquad \text{(Galerkin orthogonality)}.$$

Then for any $w \in \mathcal{V}$,

$$\|u - u_{n_b}\|_a^2 = a(u - u_{n_b}, u - w) \le \|u - u_{n_b}\|_a \, \|u - w\|_a \quad \Longrightarrow \quad \|u - u_{n_b}\|_a \le \inf_{w \in \mathcal{V}} \|u - w\|_a .$$

*So in the energy norm the Galerkin solution is not merely good, it is the best available in that space. This is the precise sense in which slide 36 is true.*

### Why a truncated Galerkin model can lose its energy balance

Multiply the reduced system by $\boldsymbol{a}^\top$ and look at the quadratic term:

$$\frac{\mathrm{d}}{\mathrm{d}t}\frac{\|\boldsymbol{a}\|^2}{2} = \boldsymbol{a}^\top \boldsymbol{c}_0 + \boldsymbol{a}^\top \boldsymbol{L}\boldsymbol{a} + \sum_{m,n,k} C_{mnk}\, a_m a_n a_k .$$

In the full system the convective term is energy conserving, so the triple sum over *all* modes vanishes. After truncation the sum runs only over $m, n, k \le n_r$ and the cancellation is no longer exact: the retained modes keep exchanging energy with modes that are no longer there. *This is the mechanism behind slide 73. The remedy is either to restore the missing dissipation with a closure term, or to enforce the cancellation directly by constructing an energy-preserving projection.*

**Blackboard economy.** The derivations above take roughly: slides 10 and 12, twelve minutes; slide 16 with the worked example, ten; slide 18, five; slide 21, five; slides 29 to 33, twenty-five; slides 40 to 45, twenty; slides 50 and 52, fifteen; slide 58, fifteen minutes. That is about an hour and three quarters of board work spread through the day, which is roughly a third of the available time. Anything more and the slides become decoration.